

ВВЕДЕНИЕ	3
1. Предпроектное исследование предметной области	6
1.1 Анализ социальных сетей и современных цифровых коммуникаций	6
1.2 Анализ существующих социальных сетей	8
1.3 Анализ целевой аудитории социальной сети	11
1.4 Формирование требований к системе	14
1.4.1 Функциональные требования	14
1.4.2 Нефункциональные требования	15
1.4.3 Требования к структуре данных	16
1.5 Постановка задачи разработки программного обеспечения	17
2. ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ	20
2.1 Выбор и обоснование архитектурного стиля	20
2.2 Моделирование архитектуры с использованием подхода С4	22
2.2.1 Диаграмма контекста системы	23
2.2.2 Диаграмма контейнеров	23
2.2.3 Диаграмма компонентов	25
2.3 Проектирование базы данных	26
2.3.1 Обоснование выбора СУБД и архитектуры хранения	26
2.3.2 Физическая модель данных	26
2.3.3 Спецификация таблиц базы данных	27
2.3.4 Хранимые процедуры, триггеры и оптимизация	29
2.3.5 Обеспечение безопасности на уровне БД	30
2.4 Проектирование пользовательского интерфейса	31
2.4.1 Концепция и визуальный стиль	31
2.4.2 Модуль аутентификации и регистрации	32
2.4.3 Структура макета и навигация	32
2.4.4 Главная страница и лента новостей	33
2.4.5 Управление профилем и пользовательским контентом	34
2.4.6 Социальные связи и файловое хранилище	35
2.5 Детальное проектирование	37
2.5.1 Диаграммы классов ключевых компонентов	37
2.5.2 Диаграммы последовательности для основных сценариев	37
2.5.3 Применение шаблонов проектирования	38
2.6 Обоснование выбора стека технологий и инструментов разработки	39
3. РАЗРАБОТКА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ СОЦИАЛЬНОЙ СЕТИ ДЛЯ СТУДЕНТОВ ВЫСШИХ УЧЕБНЫХ ЗАВЕДЕНИЙ	41
3.1 Организация процесса разработки	41
3.1.1 Выбор методологии и инструменты управления	41
3.1.2 Система контроля версий и стратегия ветвления	41
3.2 Соответствие реализации спроектированной архитектуре	42
3.3 Уровень 4 модели С4: Детализация ключевых компонентов	42
3.3.1 Реализация компонента публикаций	43
3.3.2 Реализация компонента доступа к данным	44
3.4 Демонстрация применения принципов качественного кода	44
3.4.1 Принципы SOLID	44
3.4.2 Принципы чистого кода и отладка	45
3.5 Интеграция с внешними библиотеками	45
3.6 Обеспечение безопасности и обработка ошибок	46
3.6.1 Аутентификация и авторизация	46
3.6.2 Валидация входных данных	46
3.6.3 Обработка исключений	46
3.7 Версионность и управление конфигурацией	47

4. ТЕСТИРОВАНИЕ, ОБЕСПЕЧЕНИЕ КАЧЕСТВА И СОПРОВОЖДЕНИЕ ПРОГРАММНОГО ПРОДУКТА	49
4.1 Стратегия тестирования программного продукта	49
4.1.1 Уровни и типы тестирования	49
4.1.2 Инструментарий тестирования	49
4.2 Модульное тестирование	50
4.2.1 Организация тестов Backend-части	50
4.2.2 Анализ покрытия кода	51
4.3 Интеграционное и системное тестирование	52
4.3.1 Тестирование REST API и интеграция с БД	52
4.3.2 Сквозное тестирование	52
4.3.3 Нагрузочное тестирование	53
4.4 Результаты тестирования и анализ дефектов	54
4.5 Подготовка к сопровождению	55
4.5.1 Процесс развертывания и CI/CD	55
4.5.2 Пользовательская и техническая документация	56
5. ТЕХНИКО-ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ ПРОЕКТА	58
5.1 Планирование ресурсов и расчет трудоемкости разработки	58
5.2 Расчет себестоимости разработки программного продукта	59
5.3 Оценка экономической эффективности проекта	60
5.4 Основные технико-экономические показатели проекта	61
5.5 Обоснование экономической целесообразности разработки	62
6. БЕЗОПАСНОСТЬ И ОХРАНА ТРУДА ПРИ ЭКСПЛУАТАЦИИ ПО	64
6.1 Анализ условий труда и эргономика рабочего места разработчика	64
6.2 Электробезопасность и пожарная безопасность	64
6.3 Информационная безопасность и защита персональных данных	65
ЗАКЛЮЧЕНИЕ	67
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	69
ПРИЛОЖЕНИЕ А	71
ПРИЛОЖЕНИЕ Б. Руководство пользователя программного продукта	74

ВВЕДЕНИЕ

В современном обществе информационные технологии занимают ключевую роль практически во всех сферах человеческой деятельности. Особенно активно цифровые технологии применяются в сфере коммуникации и обмена информацией. Одним из наиболее популярных инструментов онлайн-взаимодействия являются социальные сети, которые позволяют пользователям обмениваться сообщениями, публиковать контент, создавать сообщества и поддерживать социальные связи независимо от географического положения. [21]

Социальные сети стали неотъемлемой частью жизни молодежи, в том числе студентов высших учебных заведений и колледжей. Они используются не только для общения, но и для совместной учебной деятельности, обмена учебными материалами, обсуждения образовательных вопросов и организации различных студенческих мероприятий. Несмотря на большое количество существующих социальных платформ, большинство из них ориентировано на массовую аудиторию и не учитывает специфические потребности студенческой среды.

В связи с этим возникает необходимость создания специализированных цифровых платформ, ориентированных именно на студентов образовательных учреждений. Такая социальная сеть может объединять обучающихся в едином информационном пространстве, облегчать коммуникацию между студентами различных курсов и направлений подготовки, а также способствовать обмену знаниями и учебными материалами.

Разработка социальной сети для студентов вузов и колледжей позволит создать удобную среду для общения, взаимодействия и совместной деятельности. Подобная система может включать в себя функции публикации новостей, обмена сообщениями, создания тематических групп, а также инструменты для поиска однокурсников и единомышленников.

Актуальность темы данной выпускной квалификационной работы обусловлена возрастающей ролью цифровых коммуникаций в образовательной с

еде и необходимостью создания специализированных информационных систем, обеспечивающих эффективное взаимодействие студентов. Создание такой платформы позволит повысить уровень коммуникации между обучающимися и обеспечить более удобный доступ к учебной и организационной информации.

Целью данной выпускной квалификационной работы является разработка программного обеспечения социальной сети для студентов высших учебных заведений, предназначенной для студентов высших учебных заведений и колледжей, обеспечивающей удобное взаимодействие пользователей, обмен информацией и создание студенческих сообществ.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Провести анализ предметной области и существующих социальных сетей.
2. Определить основные требования к разрабатываемой системе.
3. Спроектировать архитектуру социальной сети.
4. Разработать структуру базы данных системы.
5. Реализовать основные функциональные модули социальной сети.
6. Разработать пользовательский интерфейс приложения.
7. Провести тестирование разработанного программного обеспечения.
8. Провести анализ экономической ценности разработки.
9. Проанализировать безопасность при работе с приложением.

Объектом исследования является процесс онлайн-коммуникации студентов в цифровой среде.

Предметом исследования является разработка программной системы социальной сети, предназначенной для организации взаимодействия студентов.

Практическая значимость работы заключается в разработке прототипа социальной сети для студентов вузов и колледжей, который может быть использован в качестве платформы для общения, обмена информацией и создания студенческих сообществ.

Результатом выполнения выпускной квалификационной работы является программная система социальной сети, реализующая основные функции взаимодействия пользователей и обеспечивающая удобную работу студентов в едином информационном пространстве.

1. Предпроектное исследование предметной области

1.1 Анализ социальных сетей и современных цифровых коммуникаций

В последние десятилетия развитие информационных технологий привело к значительным изменениям в способах коммуникации между людьми. Интернет стал основным инструментом обмена информацией, а цифровые платформы позволили пользователям взаимодействовать друг с другом независимо от географического расстояния. Одним из наиболее популярных и востребованных инструментов цифровой коммуникации стали социальные сети. [21]

Социальные сети представляют собой интернет-платформы, предназначенные для создания и поддержания социальных связей между пользователями. Они предоставляют возможности для обмена текстовой, графической и мультимедийной информацией, организации групп по интересам, публикации новостей, а также для осуществления личного и группового общения.

Основной особенностью социальных сетей является возможность формирования виртуальных сообществ пользователей, объединённых общими интересами, профессиональной деятельностью или образовательной средой. Благодаря этому социальные сети стали важным инструментом коммуникации как в личной, так и в профессиональной сфере.

Современные социальные сети выполняют широкий спектр функций. К основным из них относятся:

- создание личного профиля пользователя;
- добавление друзей и подписчиков;
- обмен личными сообщениями;
- публикация текстовых записей, фотографий и видеоматериалов;
- формирование тематических сообществ и групп;
- организация мероприятий;
- обмен файлами и ссылками.

Благодаря данным функциям социальные сети становятся универсальной платформой для взаимодействия между людьми. Пользователи могут не только поддерживать личные связи, но и участвовать в обсуждении различных тем, обмениваться знаниями и получать актуальную информацию.

Особенно активными пользователями социальных сетей являются молодые люди, в том числе студенты высших учебных заведений и колледжей. Для данной категории пользователей социальные сети выполняют не только развлекательную, но и образовательную функцию. Студенты используют их для общения с однокурсниками, обсуждения учебных заданий, обмена учебными материалами и координации совместной работы над проектами.

Развитие цифровых коммуникаций также связано с распространением мобильных устройств и высокоскоростного интернета. Современные пользователи получают доступ к социальным сетям с помощью смартфонов, планшетов и ноутбуков, что обеспечивает постоянное присутствие в цифровом информационном пространстве. Это значительно повышает скорость обмена информацией и делает коммуникацию более оперативной.

Кроме того, современные социальные сети активно используют технологии персонализации контента. На основе анализа интересов и активности пользователей формируются индивидуальные новостные ленты, позволяющие отображать наиболее релевантную информацию. Такой подход повышает удобство использования платформ и способствует увеличению вовлеченности пользователей.

Несмотря на широкое распространение универсальных социальных сетей, таких как платформы массового общения, в последние годы наблюдается тенденция к созданию специализированных социальных платформ, ориентированных на определённые категории пользователей. К ним относятся профессиональные социальные сети, образовательные

платформы, а также специализированные сообщества для студентов и преподавателей.

Для студентов образовательных учреждений цифровые коммуникации играют особую роль. В рамках учебного процесса им необходимо регулярно обмениваться информацией, обсуждать учебные задания, делиться материалами и организовывать совместную деятельность. Использование специализированной социальной сети может значительно упростить данные процессы и создать удобную среду для взаимодействия.

Таким образом, социальные сети являются важным элементом современной цифровой коммуникации. Они обеспечивают эффективное взаимодействие пользователей, позволяют обмениваться информацией и формировать онлайн-сообщества. Развитие технологий и рост потребностей пользователей способствуют появлению новых форм цифровых платформ, ориентированных на конкретные социальные группы.

В связи с этим разработка социальной сети, предназначенной для студентов вузов и колледжей, является актуальной задачей. Такая система может обеспечить удобное пространство для общения, обмена знаниями и организации совместной учебной деятельности.

1.2 Анализ существующий социальных сетей

В настоящее время существует большое количество социальных сетей, предназначенных для общения пользователей, обмена информацией и формирования онлайн-сообществ. Они отличаются функциональными возможностями, аудиторией пользователей и назначением. Для определения требований к разрабатываемой социальной сети необходимо провести анализ наиболее популярных платформ, используемых студентами. [19]

Рассмотрим наиболее распространённые социальные сети и коммуникационные платформы.

Одной из наиболее популярных социальных сетей является ВКонтакте. Данная платформа предоставляет пользователям широкий набор функций для общения и обмена информацией. Пользователи могут создавать личные профили, добавлять друзей, публиковать записи на своей странице, создавать сообщества и обмениваться личными сообщениями. Кроме того, платформа поддерживает публикацию фотографий, видеозаписей и различных мультимедийных материалов.

Социальная сеть ВКонтакте широко используется студентами для общения, организации студенческих сообществ и обмена учебными материалами. Однако данная платформа ориентирована на широкую аудиторию пользователей и не предоставляет специализированных инструментов для образовательного взаимодействия.

Другой популярной платформой для общения является Telegram. Данный сервис представляет собой мессенджер, позволяющий пользователям обмениваться текстовыми сообщениями, файлами, изображениями и видеоматериалами. Одним из основных преимуществ платформы является высокая скорость передачи сообщений, а также возможность создания групповых чатов и каналов.

Платформа Telegram часто используется студентами для организации учебных чатов, обмена конспектами и обсуждения учебных заданий. Тем не менее данный сервис ориентирован преимущественно на обмен сообщениями и не обладает полноценным функционалом социальной сети, таким как персонализированная новостная лента или система пользовательских профилей с расширенными возможностями.

Ещё одной популярной платформой для онлайн-коммуникации является Discord. Изначально данный сервис был разработан для общения игроков, однако в настоящее время он активно используется различными сообществами, включая образовательные. Платформа предоставляет возможность создания серверов, внутри которых пользователи могут организовывать текстовые и голосовые каналы для общения.

Преимуществом платформы Discord является удобная система групповой коммуникации и возможность проведения голосовых конференций. Однако интерфейс сервиса может быть сложным для новых пользователей, а основная концепция платформы ориентирована на голосовое и текстовое общение, а не на создание полноценной социальной сети.

Кроме универсальных социальных сетей существуют также специализированные образовательные платформы, предназначенные для организации учебного процесса. К таким системам относятся различные системы дистанционного обучения и образовательные порталы. Они позволяют преподавателям размещать учебные материалы, проводить тестирование и организовывать взаимодействие со студентами.

Однако подобные платформы чаще всего используются исключительно в рамках образовательного процесса и не предназначены для свободного общения студентов. Их функциональность ограничена задачами обучения и не включает возможности формирования социальных связей между пользователями.

На основе проведённого анализа можно выделить основные преимущества и недостатки существующих платформ. Для наглядности результаты анализа представлены в таблице 1.1.

Таблица 1.1 - Сравнительная таблица существующих социальных сетей

Платформа	Основные функции	Преимущества	Недостатки
ВКонтакте	Социальная сеть, публикации, группы, сообщения	группы, сообщения широкая аудитория, разнообразный функционал	отсутствие специализации для студентов
Telegram	обмен сообщениями, групповые чаты, каналы	высокая скорость работы, удобный обмен файлами	ограниченный функционал социальной сети

Продолжение таблицы 1.1

Discord	текстовые и голосовые каналы, серверы	удобная групповая коммуникация	сложность интерфейса, ориентированность на голосовое общение
Образовательные платформы	размещение учебных материалов, тестирование	поддержка учебного процесса	ограниченные возможности социальной коммуникации

Проведённый анализ показывает, что существующие платформы обладают широкими возможностями коммуникации, однако ни одна из них не ориентирована исключительно на потребности студентов вузов и колледжей. Универсальные социальные сети предназначены для массовой аудитории, а образовательные системы не обеспечивают полноценное социальное взаимодействие между пользователями.

Таким образом, существует необходимость разработки специализированной социальной сети, предназначенной для студентов образовательных учреждений. Такая система должна объединять возможности социальной сети и инструменты, полезные для учебного взаимодействия, включая обмен учебными материалами, организацию групп по интересам и удобную коммуникацию между студентами.

1.3 Анализ целевой аудитории социальной сети

При разработке любой информационной системы важным этапом является определение и анализ целевой аудитории. Целевая аудитория представляет собой группу пользователей, для которых создаётся программный продукт и чьи потребности должны быть учтены при проектировании функциональности системы. [21]

В рамках данной выпускной квалификационной работы целевой аудиторией разрабатываемой социальной сети являются студенты высших учебных заведений и колледжей. Данная категория пользователей характеризуется высокой активностью в цифровой среде и активно

использует интернет-сервисы для общения, получения информации и организации совместной деятельности.

Студенты представляют собой одну из наиболее активных групп пользователей социальных сетей. Большинство обучающихся регулярно использует цифровые платформы для обмена сообщениями, просмотра новостей, публикации материалов и участия в различных онлайн-сообществах. Использование социальных сетей для студентов является привычным способом коммуникации, который дополняет традиционные формы взаимодействия.

Особенностью студенческой аудитории является необходимость постоянного обмена информацией, связанной с образовательным процессом. Студенты регулярно взаимодействуют друг с другом для обсуждения учебных заданий, подготовки к экзаменам, обмена конспектами и учебными материалами. Кроме того, социальные сети используются для организации совместных проектов, обсуждения научных работ и координации учебной деятельности.

Другой важной особенностью данной аудитории является потребность в быстром и удобном доступе к информации. Студенты активно используют мобильные устройства, такие как смартфоны и планшеты, для доступа к интернету и коммуникации. Это означает, что разрабатываемая система должна обеспечивать удобный интерфейс и возможность использования на различных типах устройств.

Кроме образовательных задач, студенты используют социальные сети для организации внеучебной деятельности. К таким видам активности относятся участие в студенческих мероприятиях, организация встреч, поиск единомышленников и участие в различных сообществах по интересам. Социальная сеть, ориентированная на студентов, должна учитывать данные потребности и предоставлять возможности для формирования тематических групп и сообществ.

Анализ целевой аудитории позволяет выделить основные категории пользователей социальной сети:

- студенты высших учебных заведений;
- студенты колледжей и техникумов;
- студенческие организации и сообщества;
- участники учебных групп и проектных команд.

Для каждой из указанных категорий пользователей социальная сеть может выполнять различные функции. Например, студенты могут использовать систему для общения и обмена учебными материалами, а студенческие организации – для информирования участников о мероприятиях и координации деятельности.

На основе анализа потребностей целевой аудитории можно выделить основные задачи, которые должна решать разрабатываемая социальная сеть:

- обеспечение удобного общения между студентами;
- возможность обмена учебными материалами;
- создание групп по интересам и учебным дисциплинам;
- организация студенческих мероприятий;
- поиск и установление контактов между студентами различных учебных заведений.

Кроме функциональных требований важно учитывать и особенности поведения пользователей. Студенты предпочитают простые и интуитивно понятные интерфейсы, позволяющие быстро выполнять необходимые действия. Поэтому при разработке социальной сети необходимо уделить внимание удобству пользовательского интерфейса и скорости работы системы.

Таким образом, анализ целевой аудитории показывает, что студенты вузов и колледжей нуждаются в специализированной цифровой платформе, которая будет сочетать возможности социальной сети и инструменты для образовательного взаимодействия. Создание такой системы позволит упростить коммуникацию между студентами, повысить эффективность

обмена информацией и создать единое информационное пространство для студенческого сообщества.

1.4 Формирование требований к системе

После анализа предметной области, существующих социальных сетей и целевой аудитории необходимо определить требования к разрабатываемой системе. Формирование требований является важным этапом проектирования программного продукта, так как на основе данных требований определяется структура системы, её функциональные возможности и особенности реализации. [21]

Требования к системе представляют собой совокупность условий и характеристик, которым должна соответствовать разрабатываемая программная система для удовлетворения потребностей пользователей. В рамках разработки социальной сети для студентов вузов и колледжей требования можно разделить на функциональные и нефункциональные.

1.4.1 Функциональные требования

Функциональные требования определяют основные возможности системы и описывают действия, которые пользователь может выполнять в рамках социальной сети. [21]

К основным функциональным требованиям разрабатываемой системы относятся следующие:

1. Регистрация пользователей в системе. Система должна предоставлять возможность регистрации новых пользователей с использованием электронной почты или других данных.
2. Авторизация пользователей. Пользователь должен иметь возможность входа в систему с использованием своего логина и пароля.

3. Создание и редактирование профиля пользователя. Пользователь должен иметь возможность создавать личный профиль, указывать информацию о себе, учебном заведении, факультете и курсе обучения.

4. Поиск пользователей. Система должна предоставлять возможность поиска других пользователей по имени, учебному заведению или другим параметрам.

5. Добавление пользователей в список друзей или подписчиков. Пользователь должен иметь возможность устанавливать связи с другими пользователями.

6. Обмен личными сообщениями. Система должна обеспечивать возможность отправки и получения личных сообщений между пользователями.

7. Создание и публикация записей. Пользователи должны иметь возможность публиковать текстовые записи, изображения и ссылки.

8. Просмотр новостной ленты. Система должна формировать персонализированную ленту новостей, отображающую публикации друзей и сообществ.

9. Создание групп и сообществ. Пользователи должны иметь возможность создавать тематические группы, связанные с учебными дисциплинами, интересами или студенческими мероприятиями.

10. Обмен учебными материалами. Система должна обеспечивать возможность публикации и скачивания учебных файлов, конспектов и других материалов.

11. Комментирование публикаций. Пользователи должны иметь возможность оставлять комментарии к записям других пользователей.

12. Уведомления о событиях. Система должна информировать пользователей о новых сообщениях, комментариях и других событиях.

1.4.2 Нефункциональные требования

Нефункциональные требования определяют характеристики качества системы, включая её производительность, безопасность и удобство использования. [21]

К основным нефункциональным требованиям относятся:

1. Удобство использования. Интерфейс системы должен быть интуитивно понятным и удобным для пользователей.
2. Производительность. Система должна обеспечивать быструю обработку запросов и загрузку страниц.
3. Надёжность. Система должна корректно функционировать при работе большого количества пользователей.
4. Безопасность. Необходимо обеспечить защиту персональных данных пользователей и предотвращение несанкционированного доступа к системе.
5. Масштабируемость. Система должна иметь возможность расширения функциональности и увеличения количества пользователей без значительного снижения производительности.
6. Кроссплатформенность. Социальная сеть должна корректно работать на различных устройствах, включая персональные компьютеры, планшеты и смартфоны.
7. Доступность системы. Система должна обеспечивать стабильную работу и быть доступной для пользователей в любое время.

1.4.3 Требования к структуре данных

Для обеспечения функционирования системы необходимо разработать структуру базы данных, которая будет хранить информацию о пользователях, публикациях, сообщениях и группах. Основные сущности базы данных могут включать: [21]

- пользователь;
- профиль пользователя;

- публикация;
- комментарий;
- сообщение;
- группа;
- участники группы;
- учебные материалы.

Корректное проектирование структуры базы данных позволит обеспечить эффективное хранение информации и быстрый доступ к данным.

Таким образом, сформированные требования определяют основные возможности разрабатываемой социальной сети и служат основой для дальнейшего проектирования системы. На следующем этапе будет выполнена постановка задачи разработки социальной сети и определены основные этапы реализации проекта.

1.5 Постановка задачи разработки программного обеспечения

В условиях активного развития цифровых технологий и широкого распространения интернет-коммуникаций социальные сети становятся важным инструментом взаимодействия пользователей. Особенно значимую роль такие платформы играют для студентов, поскольку позволяют поддерживать постоянное общение, обмениваться информацией и организовывать совместную деятельность. [21]

Проведённый анализ существующих социальных сетей показал, что большинство популярных платформ ориентировано на массовую аудиторию и не учитывает специфические потребности студенческого сообщества. В то же время образовательные системы, используемые в учебных заведениях, предназначены преимущественно для организации учебного процесса и не предоставляют широких возможностей для неформального общения и взаимодействия студентов.

Таким образом, существует необходимость создания специализированной социальной сети, ориентированной на студентов вузов и колледжей. Такая система должна обеспечивать удобное пространство для общения пользователей, обмена учебной информацией и формирования студенческих сообществ.

Основной задачей данной выпускной квалификационной работы является разработка программной системы социальной сети, предназначенной для студентов образовательных учреждений. Разрабатываемая система должна обеспечивать эффективное взаимодействие пользователей, а также предоставлять функциональные возможности, необходимые для общения и совместной учебной деятельности.

В рамках разработки социальной сети необходимо решить следующие основные задачи:

- разработать архитектуру программной системы;
- спроектировать структуру базы данных;
- реализовать систему регистрации и авторизации пользователей;
- разработать механизм создания и редактирования пользовательских профилей;
- реализовать систему публикации записей и формирования новостной ленты;
- обеспечить возможность обмена личными сообщениями между пользователями;
- реализовать механизм создания групп и сообществ;
- обеспечить возможность публикации и обмена учебными материалами;
- разработать пользовательский интерфейс системы;
- провести тестирование разработанного программного продукта.

В результате выполнения выпускной квалификационной работы должна быть создана программная система социальной сети, обеспечивающая основные функции взаимодействия пользователей. Система

должна позволять студентам регистрироваться в приложении, создавать личные профили, находить других пользователей, публиковать записи, загружать свои файлы во встроенное хранилище и участвовать в различных обсуждениях.

Разрабатываемая социальная сеть должна быть удобной в использовании, обладать интуитивно понятным интерфейсом и обеспечивать стабильную работу при одновременном использовании несколькими пользователями. Кроме того, система должна предусматривать возможность дальнейшего расширения функциональности.

Таким образом, постановка задачи разработки социальной сети определяет основные направления проектирования и реализации программной системы. Полученные результаты анализа предметной области и сформированные требования будут использованы на следующем этапе дипломной работы – при проектировании архитектуры системы и структуры базы данных социальной сети.

Выводы

В ходе предпроектного исследования был проведен подробный анализ предметной области, современных цифровых коммуникаций и наиболее популярных существующих социальных сетей. Выявлено, что универсальные платформы ориентированы на массовую аудиторию и не учитывают специфические потребности студентов в совмещении неформального общения и учебного взаимодействия. На основе детального анализа целевой аудитории был сформирован исчерпывающий перечень функциональных (регистрация, обмен сообщениями, создание групп, публикация файлов) и нефункциональных требований. Результатом этапа стала четкая постановка задачи на разработку специализированной социальной сети, объединяющей коммуникационные и образовательные инструменты в рамках единого информационного пространства.

2. ПРОЕКТИРОВАНИЕ АРХИТЕКТУРЫ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ

2.1 Выбор и обоснование архитектурного стиля

Проектирование архитектуры является основополагающим этапом инженерного жизненного цикла создания программного обеспечения, определяющим стабильность, расширяемость и производительность будущей системы. Для специализированной социальной сети студентов Северо-Кавказского федерального университета критически важно выбрать архитектурную парадигму, обеспечивающую слабую связность модулей и независимость интерфейса от бизнес-логики. [11]

В рамках системного анализа были исследованы три альтернативных архитектурных стиля:

1. Монолитная архитектура. Данный стиль предполагает объединение пользовательского интерфейса, бизнес-логики и слоя доступа к данным в рамках единого исполняемого контекста и одной кодовой базы. Такая архитектура обладает следующими преимуществами:

- высокая скорость первоначальной разработки;
- простота локального тестирования;
- простота развертывания единого программного пакета.

Но также и обладает рядом недостатков:

- жесткая связность компонентов (tight coupling);
- усложнение проведения рефакторинга;
- невозможность независимого масштабирования отдельных высоконагруженных узлов (например, модуля обмена сообщениями);
- высокий риск полной деградации системы при отказе одного из внутренних модулей.

2. Микросервисная архитектура. Данный подход базируется на разделении единой информационной системы на комплекс автономных, слабосвязанных сервисов, каждый из которых решает изолированную бизнес-задачу (сервис авторизации, сервис публикаций, сервис чатов) и

функционирует в собственном процессе. У этого вида архитектуры есть следующие преимущества:

- технологическая независимость модулей;
- высокая отказоустойчивость;
- возможность изолированного масштабирования и независимого деплоя.

А также недостатки:

- избыточная инфраструктурная сложность;
- существенные накладные расходы на организацию межсервисного взаимодействия (IPC, сетевые задержки при RPC/REST);
- сложность обеспечения транзакционной целостности данных (Distributed Transactions) и нецелесообразность применения в проектах;
- реализуемых одним разработчиком в ограниченные сроки.

3. Клиент-серверная многоуровневая архитектура (REST API + SPA).

Архитектурный стиль, основанный на строгом разделении ответственности между клиентской частью, функционирующей как Single Page Application (SPA), и серверной частью (Backend Application), предоставляющей унифицированный программный интерфейс (REST API).

Для минимизации рисков и обеспечения гибкости разработки был проведен критериальный анализ альтернатив по методу взвешенных оценок. Результаты анализа сведены в таблицу 2.1.

Таблица 2.1 – Сравнительный анализ архитектурных стилей

Критерий сравнения	Монолитная архитектура	Микросервисная архитектура	Клиент-серверная архитектура (REST API + SPA)
Скорость индивидуальной разработки	Высокая	Низкая	Высокая
Изоляция отказов компонентов	Низкая	Высокая	Средняя
Простота развертывания	Высокая	Низкая	Средняя

Продолжение таблицы 2.1

Возможность кроссплатформенного развития	Низкая	Высокая	Высокая
Накладные расходы на инфраструктура	Минимальные	Высокие	Средние

На основании проведенного анализа для разработки студенческой социальной сети был выбран клиент-серверный многоуровневый архитектурный стиль на основе REST API. Данное инженерное решение обосновано следующими факторами:

1. Принцип разделения ответственности (Separation of Concerns): Серверная часть полностью изолирует бизнес-логику и управление реляционной базой данных, пресекая прямое влияние интерфейсных изменений на целостность данных. Клиентская часть (SPA) берет на себя вычислительную нагрузку по рендерингу интерфейса и управлению состоянием на устройстве пользователя.

2. Потенциал кроссплатформенной интеграции: Разрабатываемый бэкенд становится универсальным поставщиком данных. В случае необходимости расширения проекта до мобильных платформ (iOS, Android), существующий REST API может быть повторно использован без модификации серверного кода.

3. Оптимизация под ресурсы одного инженера: Архитектура позволяет последовательно вести разработку: спроектировать и протестировать программные эндпоинты, после чего перейти к конструированию интерфейса.

2.2 Моделирование архитектуры с использованием подхода C4

Для формализации и документирования структуры создаваемой социальной сети применяется методология архитектурного моделирования C4 model. Данный подход позволяет уйти от перегруженных и неструктурированных UML-схем в пользу иерархического представления

системы, разделенного на четыре уровня абстракции: контекст, контейнеры, компоненты и код. [7]

2.2.1 Диаграмма контекста системы

На уровне контекста (System Context) разрабатываемый программный продукт позиционируется как структурированный «черный ящик». Диаграмма определяет внешние границы системы, выявляет ключевых акторов и их взаимодействие с сопутствующим окружением. [7]

Основным актором выступает Студент – авторизованный или неавторизованный пользователь образовательного учреждения, решающий коммуникативные и учебные задачи посредством системы. В качестве внешней зависимой системы ИТКС задействован Внешний SMTP-сервер (например, Mail.ru или Яндекс.Почта), необходимый для трансляции писем верификации при регистрации и сбросе паролей пользователей.



Рисунок 2.1 – Диаграмма контекста системы

2.2.2 Диаграмма контейнеров

Диаграмма контейнеров (Container Diagram) осуществляет декомпозицию «черного ящика» на обособленные, самостоятельно

развертываемые программные элементы (контейнеры), фиксируя используемые технологии и протоколы сетевого взаимодействия. [7]

В рамках проектируемой социальной сети выделяются три ключевых контейнера:

1. Frontend SPA (React/TypeScript): Инкапсулирует в себе пользовательский интерфейс. Контейнер загружается в браузер клиента, выполняется на стороне его устройства и отправляет структурированные запросы к серверу.

2. Backend Web API (FastAPI/Python): Выступает в роли ядра системы. Обрабатывает входящие HTTP-запросы, выполняет аутентификацию, валидацию и реализует ключевые алгоритмы бизнес-логики.

3. Database Storage (PostgreSQL): Обеспечивает персистентное (энергонезависимое) хранение всей информации социальной сети, поддерживая транзакционную целостность.

Сетевой обмен между Frontend SPA и Backend Web API организуется по протоколу HTTPS с использованием архитектурного принципа REST и передачей полезной нагрузки в формате JSON. Взаимодействие бэкенда с СУБД PostgreSQL выполняется по стандартному бинарному протоколу драйвера базы данных с использованием пула соединений.

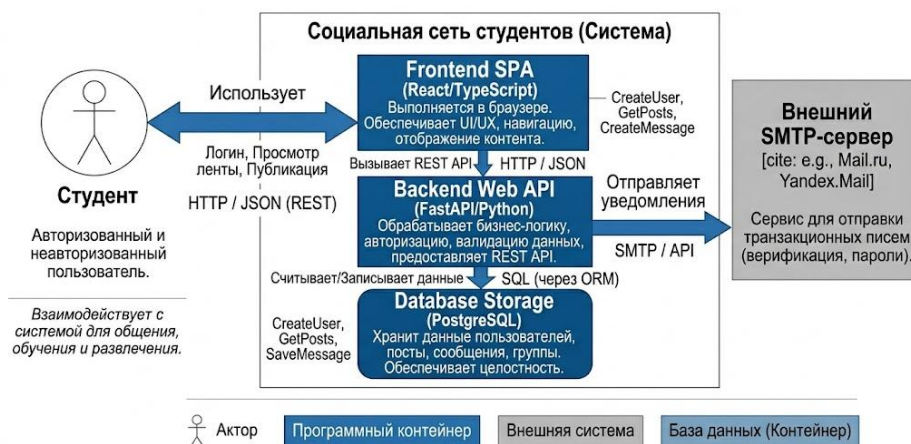


Рисунок 2.2 – Диаграмма контейнеров системы

2.2.3 Диаграмма компонентов

Диаграмма компонентов (Component Diagram) детализирует внутреннее устройство контейнера Backend Web API, демонстрируя его разделение на логические модули и слои (рисунок 2.3). На данном уровне применяется классическая слоистая архитектура (Layered Architecture): [7]

1. Слой API Controllers (Контроллеры): Отвечает за маршрутизацию входящих URL, перехват HTTP-запросов, десериализацию входных JSON-объектов и первичную валидацию полей. Выделяются AuthController (управление сессиями), PostController (работа с публикациями), MessageController (чат-система) и GroupController (управление сообществами).

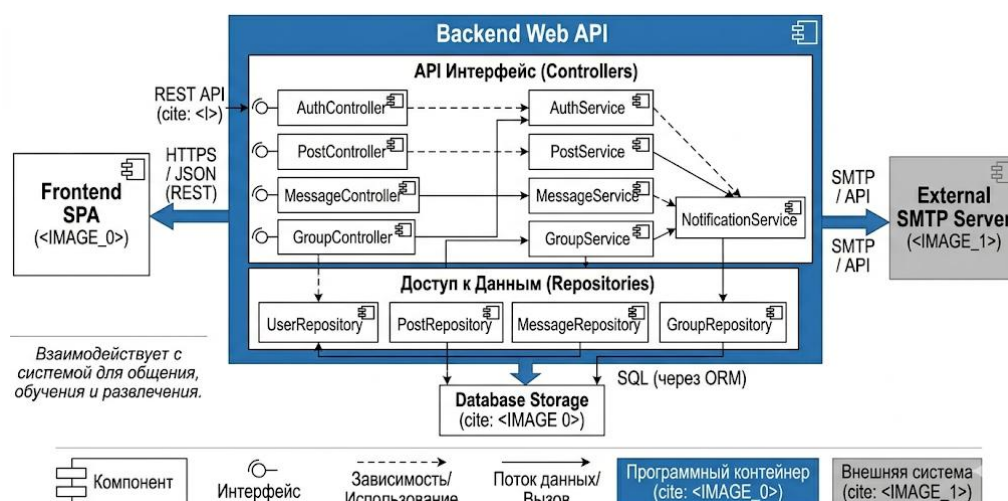


Рисунок 2.3 – Диаграмма компонентов Backend Web API

2. Слой Business Logic Services (Сервисы): Компоненты, содержащие чистую бизнес-логику системы без привязки к транспортному протоколу HTTP. Контроллеры делегируют выполнение задач соответствующим сервисам: *AuthService*, *PostService*, *MessageService*.

3. Слой Data Access Repositories (Репозитории): Компоненты, реализующие паттерн проектирования Repository. Они изолируют SQL-

запросы к базе данных, абстрагируя вышележащие слои от специфики конкретной СУБД.

2.3 Проектирование базы данных

2.3.1 Обоснование выбора СУБД и архитектуры хранения

Для реализации функциональных требований к социальной сети студентов была выбрана реляционная система управления базами данных PostgreSQL. Выбор обусловлен необходимостью строгого соблюдения целостности данных, поддержки сложных связей между сущностями (пользователи, посты, друзья, комментарии) и высокой производительностью при конкурентном доступе. [14]

В качестве платформы разработки используется Supabase (Backend-as-a-Service на базе PostgreSQL), что позволяет использовать встроенные механизмы аутентификации (auth.users), объектного хранения (Storage) и, самое главное, реализовать логику разграничения доступа непосредственно на уровне БД с помощью механизма Row Level Security (RLS).

2.3.2 Физическая модель данных

Физическая архитектура базы данных построена на принципе расширения стандартной схемы аутентификации Supabase. Центральной сущностью является таблица profiles (публичные профили), которая находится в отношении 1:1 с системной таблицей auth.users. [14]

Спроектированная структура таблиц и связей между ними представлена на текстовой ER-диаграмме (Рисунок 2.4).

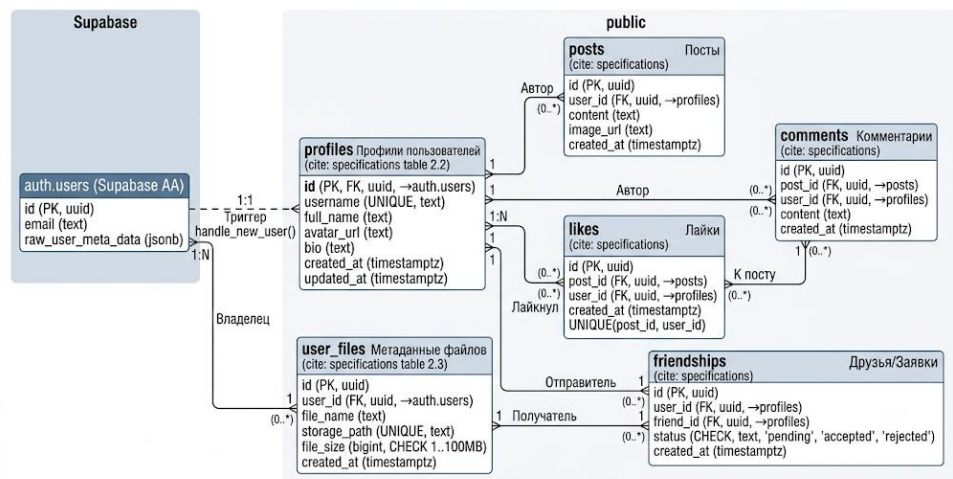


Рисунок 2.4 – Схема связей в базе данных социальной сети

При проектировании была проведена нормализация данных до третьей нормальной формы (3НФ), что минимизирует избыточность и исключает аномалии обновления.

2.3.3 Спецификация таблиц базы данных

В данном подразделе приведена детальная спецификация разработанных таблиц БД с указанием типов данных PostgreSQL, ограничений (Constraints) и внешних ключей (FK). Для всех первичных ключей (PK) используется тип `uuid`, генерируемый автоматически функцией `gen_random_uuid()`, что обеспечивает уникальность ID даже при масштабировании системы. [14]

1. Таблица `profiles` (Профили пользователей)

Является расширением базовой таблицы `auth.users`. Данные сюда заносятся автоматически при регистрации пользователя через триггер. Спецификация представлена в Таблице 2.2.

Таблица 2.2 – Спецификация таблицы profiles

Поле	Тип	Ограничения	Описание
id	uuid	PK, FK -> auth.users(id), ON DELETE DELETE	Уникальный ID пользователя
username	text	UNIQUE, NOT NULL	Уникальный никнейм
full_name	text	NOT NULL	Полное имя пользователя
avatar_url	text	DEFAULT ""	URL аватара
bio	text	DEFAULT ""	Описание “О себе”
created_at	timestampz	DEFAULT now()	Дата создания профиля
updated_at	timestampz	DEFAULT now()	Дата обновления профиля

2. Таблицы пользовательского контента (posts, comments, likes)

Эти таблицы реализуют основную функциональность социальной активности. Во всех таблицах настроено каскадное удаление (ON DELETE CASCADE) для соблюдения целостности данных при удалении пользователя или поста:

- posts (Посты): Хранит тексты публикаций и URL прикрепленных изображений. Автор поста определяется внешним ключом user_id → profiles(id). Основное поле — content (text, not null);
- comments (Комментарии): Связывает пользователя (user_id) с конкретным постом (post_id). Хранит текст комментария (content);
- likes (Лайки): Реализует отношение многие-ко-многим. Содержит составное уникальное ограничение UNIQUE(post_id, user_id), что гарантирует возможность поставить только один лайк на один пост от одного пользователя.

3. Таблица friendships (Друзья/Заявки)

Реализует сложную логику социальных связей с использованием графовой структуры. Таблица содержит два внешних ключа на одну и ту же таблицу profiles: user_id (отправитель заявки) и friend_id (получатель).

Логика целостности обеспечивается следующими ограничениями:

- UNIQUE(user_id, friend_id): исключает дублирование заявок между одной парой пользователей.
- CHECK (user_id != friend_id): проверочное ограничение, запрещающее пользователю отправлять заявку в друзья самому себе.
- Поле status (text) принимает строго определенные значения: 'pending', 'accepted', 'rejected', что контролируется проверочным ограничением CHECK IN.

4. Таблица user_files

Хранит метаданные файлов пользователей, загруженных в приватный бакет Supabase Storage. Эта таблица необходима для организации доступа к файлам на уровне БД. Спецификация приведена в Таблице 2.3.

Таблица 2.3 – Спецификация таблицы user_files

Поле	Тип	Ограничения	Описание
id	uuid	PD, DEFAULT gen_random_uuid()	ID файла
user_id	uuid	FK -> auth.users(id), ON DELETE CASCADE	Владелец файла
file_name	text	NOT NULL	Имя файла
storage_path	text	UNIQUE, NOT NULL	Уникальный путь в Supabase Storage
file_size	bigint	NOT NULL, CHECK (1..104857600)	Размер в байтах (ограничение до 100 МБ)
mime_type	text		Тип файла
created_at	timestampz	DEFAULT now()	Дата загрузки

2.3.4 Хранимые процедуры, триггеры и оптимизация

Для автоматизации рутинных операций и поддержания консистентности данных в базе реализован комплекс триггеров и функций на языке PL/pgSQL. [18]

Автоматическое создание профиля: Реализована функция handle_new_user(), которая срабатывает через триггер on_auth_user_created типа AFTER INSERT на системной таблице auth.users. Функция

автоматически извлекает метаданные (username, full_name), переданные при регистрации, и создает соответствующую запись в таблице public.profiles, гарантируя 1:1 связь.

Оптимизация производительности: Для ускорения выборок, критичных для производительности социальной сети, созданы индексы на внешних ключах и полях дат. Основные индексы:

1. Индексы friendships_user_id_idx и friendships_friend_id_idx для быстрой выборки списков друзей и входящих заявок.
2. Индексы по дате создания в обратном порядке (created_at DESC) в таблицах posts и user_files для оптимизации отображения ленты и списков файлов.

2.3.5 Обеспечение безопасности на уровне БД

Центральным элементом безопасности приложения является механизм Row Level Security (RLS) PostgreSQL, который заменяет традиционную middleware логику бэкенда логикой, исполняемой непосредственно в СУБД. [5]

Private Storage Bucket: Файлы пользователей хранятся в непубличном бакете private-files. Доступ к объектам в хранилище осуществляется по пути {user_id}/{filename}. Политики RLS на уровне бакета гарантируют, что читать, загружать и удалять объекты может только авторизованный владелец (auth.uid() = user_id).

Для всех таблиц в схеме public также включен RLS. Сводная стратегия политик доступа приведена в Таблице 2.4.

Таблица 2.4 – Сводная стратегия RLS-политик доступа к данным

Таблица	Операция SELECT	Операция INSERT	Операция UPDATE	Операция DELETE
profiles	Видят все пользователи	Триггер (от имени системы)	Только владелец профиля	–
posts	Видят все пользователи	Авторизованный пользователь	Только автор поста	Только автор поста
comments	Видят все пользователи	Любой авторизованный пользователь	–	Только автор комментария
likes	Видят все пользователи	Только владелец лайка	–	Только владелец лайка
friendships	Только участники (sender / receiver)	Только отправитель заявки	Только получатель заявки	Любой из участников
user_files	Только владелец файла	Только владелец файла	–	Только владелец файла

Данная архитектура безопасности обеспечивает надежное разграничение доступа к конфиденциальным данным (приватные файлы, личные диалоги), полностью перенося логику авторизации со стороны клиента или промежуточного API на уровень СУБД, что отвечает высоким стандартам безопасности современных веб-приложений.

2.4 Проектирование пользовательского интерфейса

2.4.1 Концепция и визуальный стиль

Проектирование пользовательского интерфейса (UI) клиентской части приложения выполнялось с упором на минимализм, интуитивную понятность и снижение когнитивной нагрузки на пользователя. В качестве основного паттерна проектирования выбран карточный интерфейс (Card-based UI), который позволяет четко разграничивать смысловые блоки информации на экране. [19]

Цветовая схема приложения построена на использовании спокойного фиолетово-синего градиентного фона в сочетании с контрастными белыми карточками контента. Это обеспечивает хорошую читаемость текста и

акцентирует внимание на пользовательских данных. Для элементов взаимодействия (кнопки, активные ссылки) используется насыщенный фиолетовый цвет (Primary Color), обозначающий готовность к действию.

2.4.2 Модуль аутентификации и регистрации

Экраны входа и регистрации спроектированы в виде модальных центрированных карточек, фокусирующих пользователя на единственной задаче — авторизации в системе. [19]

1. Экран входа содержит минимально необходимый набор полей: электронная почта и пароль, а также акцентную кнопку «Войти».

2. Экран регистрации расширен полями «Имя пользователя» (логин для системы) и «Полное имя» (для отображения в профиле).

Для удобства навигации между этими экранами предусмотрены текстовые ссылки быстрого перехода («Нет аккаунта? Зарегистрируйтесь» и «Есть аккаунт? Войдите»).

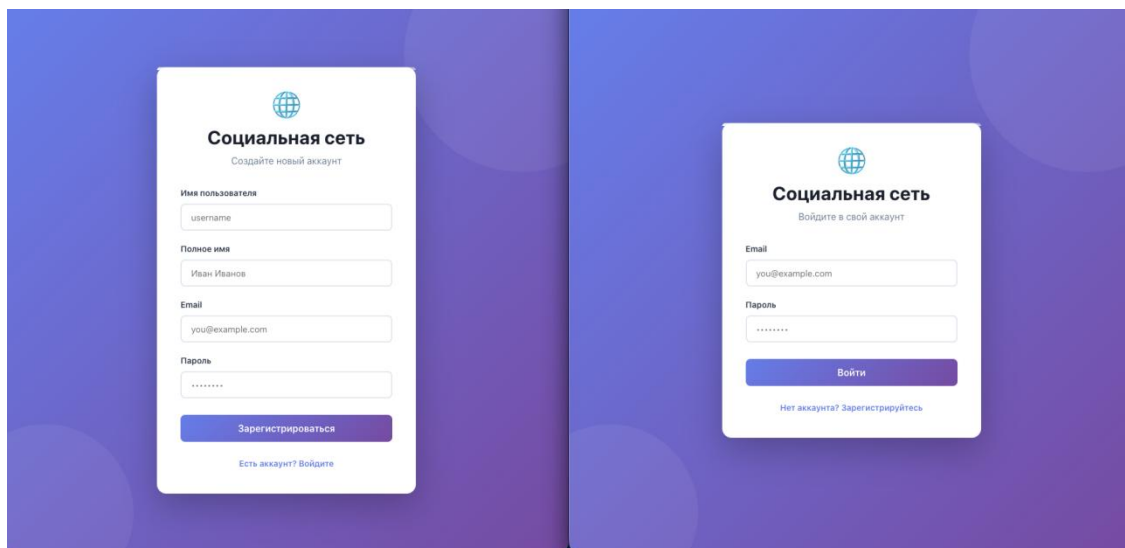


Рисунок 2.5 – Интерфейс экранов авторизации и регистрации

2.4.3 Структура макета и навигация

В авторизованной зоне приложения используется классический для веб-приложений макет с закреплённой верхней панелью навигации (Header). Панель навигации доступна с любого экрана и содержит: [19]

- логотип и название платформы;
- центральное меню с основными разделами: «Лента», «Друзья», «Мои посты», «Мои файлы», «Профиль»;
- блок пользователя в правой части (текущий никнейм и кнопка «Выйти»).

Такая структура обеспечивает быстрый доступ ко всем ключевым модулям системы в один клик.

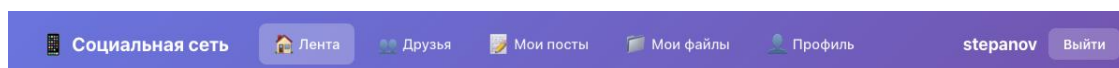


Рисунок 2.6 – Верхняя панель навигации

2.4.4 Главная страница и лента новостей

Основное рабочее пространство (раздел «Лента») спроектировано по принципу трехколоночной модульной сетки, что позволяет эффективно использовать ширину экрана: [19]

1. Левая колонка: Выполняет роль панели быстрого доступа. Содержит мини-карточку текущего профиля (аватар, имя, кнопка перехода в полный профиль), информационную подсказку и дублирующее меню разделов.

2. Центральная колонка: Основной блок взаимодействия. В верхней части расположена форма «Создать пост» с текстовым полем и кнопкой публикации. Ниже располагается бесконечная лента публикаций пользователей. Каждая публикация оформлена в виде карточки, содержащей данные автора, текст поста и интерактивные счетчики реакций (лайки и комментарии).

3. Правая колонка: Отведена под виджеты статистики и рекомендаций (например, «Лента сегодня» с подсчетом постов и лайков за сутки, а также блок «Интересное»).

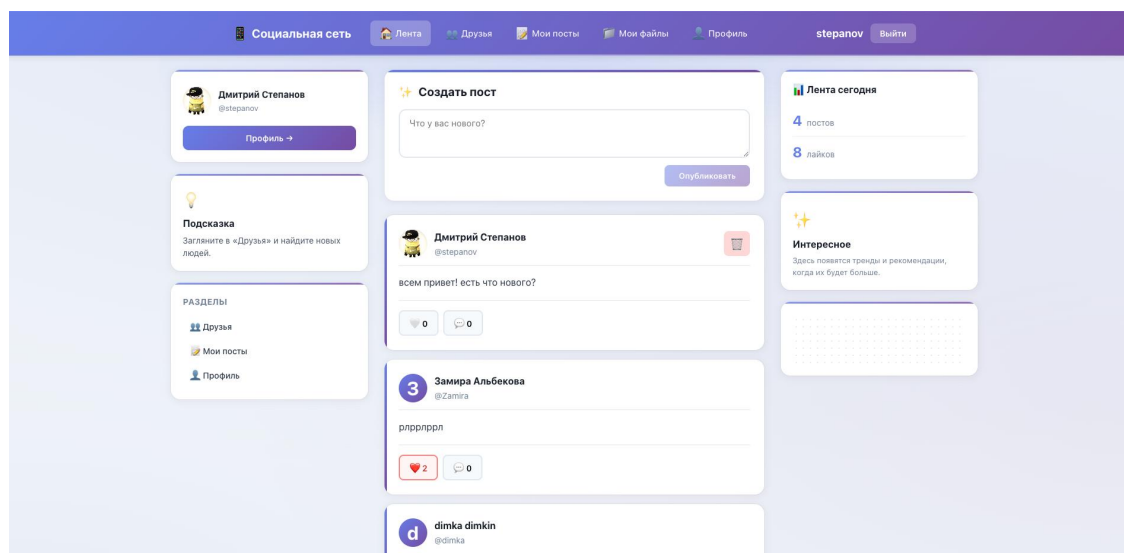


Рисунок 2.7 – Интерфейс главной страницы и ленты новостей

2.4.5 Управление профилем и пользовательским контентом

Раздел «Профиль» предоставляет подробную информацию об аккаунте. На странице размещена крупная карточка пользователя с аватаром, полным именем, уникальным идентификатором (@username), текстовым блоком «О себе» и датой регистрации. Ниже представлены блоки количественной статистики (число постов, друзей и лайков) и техническая «Информация об аккаунте». [19]

Интерфейс редактирования профиля открывается в этом же окне и позволяет гибко изменять личные данные и URL-адрес фотографии.

Для управления собственным контентом выделен раздел «Мои посты». Он отличается от общей ленты наличием расширенной статистики по личному аккаунту и элементов управления публикациями: к каждой карточке поста добавлены функциональные кнопки «Редактировать» и «Удалить».

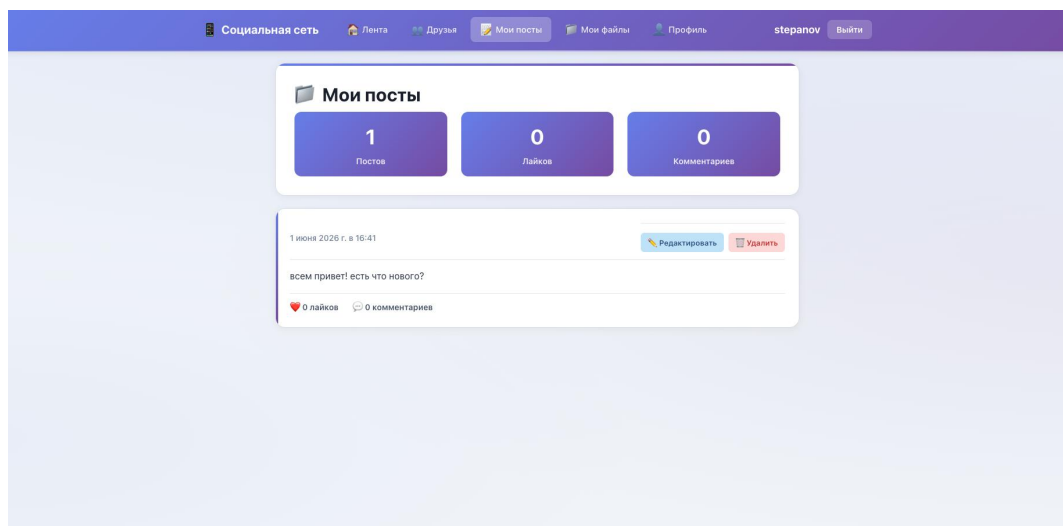


Рисунок 2.8 – Интерфейс раздела управления публикациями

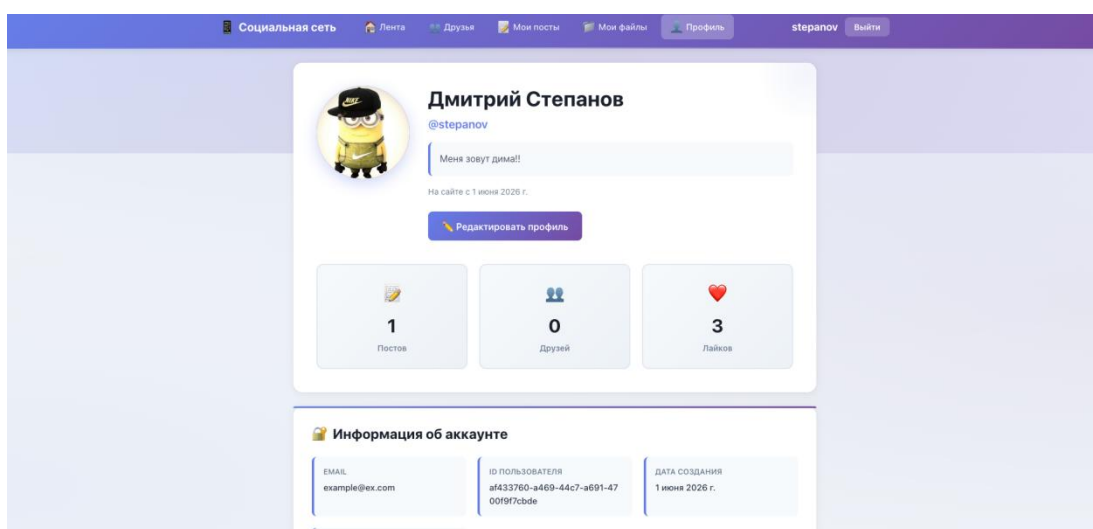


Рисунок 2.9 – Интерфейс личного профиля

2.4.6 Социальные связи и файловое хранилище

Раздел «Друзья»: Спроектирован с акцентом на визуальный поиск. Верхнюю часть экрана занимает крупная строка поиска по имени или username. Результаты выдачи и рекомендации представлены в виде плиточной сетки (Grid) карточек пользователей. На каждой карточке предусмотрена кнопка отправки запроса («Добавить в друзья»), которая меняет свое состояние на «Заявка отправлена» после нажатия, информируя пользователя об успешном действии. [19]

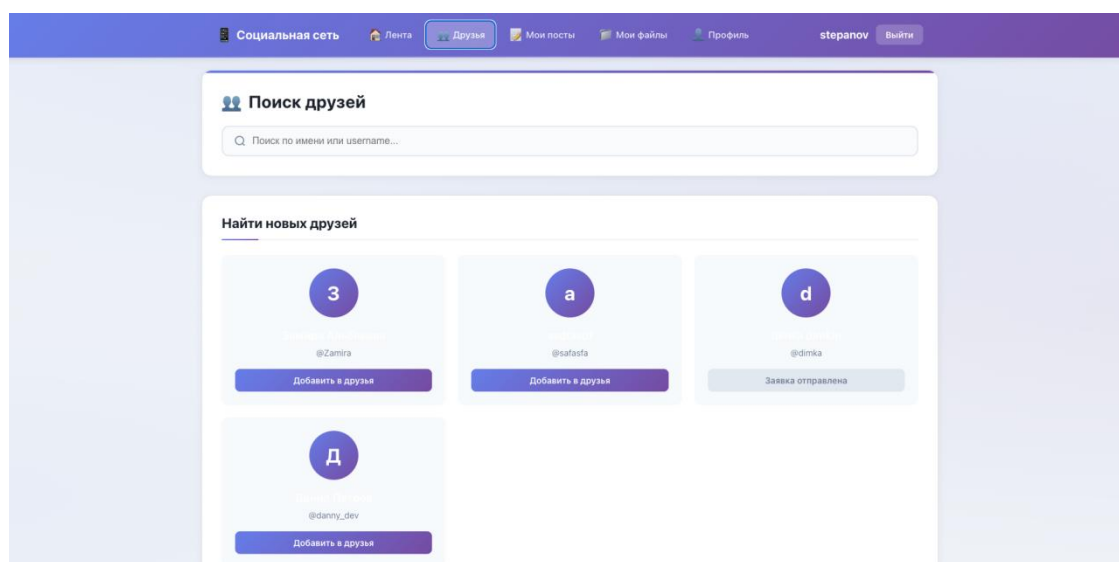


Рисунок 2.10 – Интерфейс поиска и добавления друзей

Раздел «Мои файлы»: Представляет собой интерфейс изолированного облачного хранилища. Для загрузки документов реализована интуитивно понятная зона Drag-and-Drop («Перетащите файлы сюда или нажмите для выбора») с четким указанием поддерживаемых форматов (DOCX, PDF, XLSX и др.) и лимита размера (до 100 МБ). Загруженные файлы отображаются в виде списка, где каждая запись содержит иконку типа файла, название, размер, точную дату загрузки и элементы управления (скачивание и удаление).

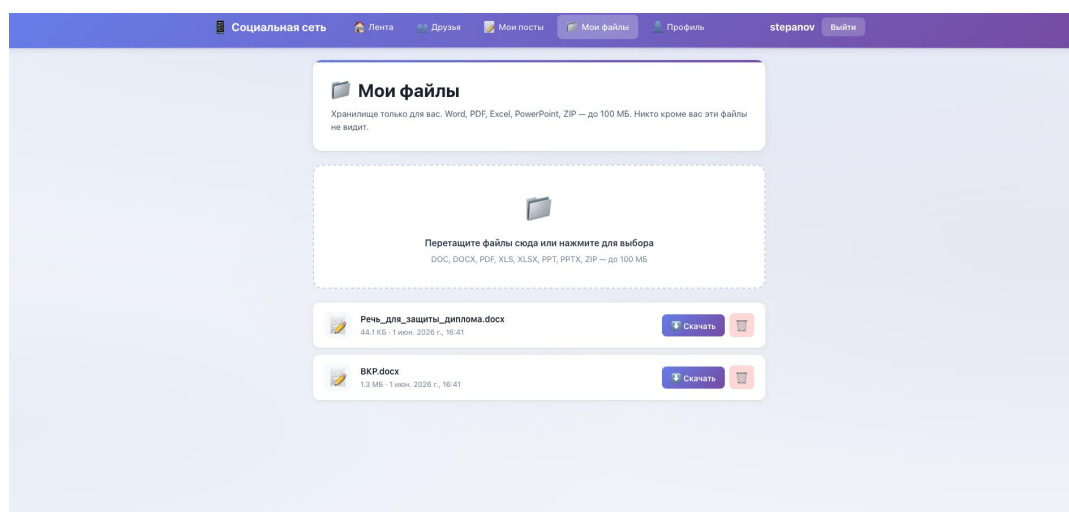


Рисунок 2.11 – Интерфейс управления файловым хранилищем

2.5 Детальное проектирование

2.5.1 Диаграммы классов ключевых компонентов

На четвертом уровне модели C4 (уровень кода) осуществляется спецификация внутренней объектной структуры программных компонентов системы. Для объектно-ориентированного проектирования модуля управления публикациями (Posts) была построена диаграмма классов UML. [24]

Диаграмма иллюстрирует изоляцию зависимостей через инверсию управления. Класс `PostController` зависит исключительно от абстрактного интерфейса `IPostService`, а конкретная реализация `PostService` взаимодействует со слоем данных через интерфейс `IPostRepository`. Это позволяет гибко подменять компоненты (например, при модульном тестировании с использованием мок-объектов).

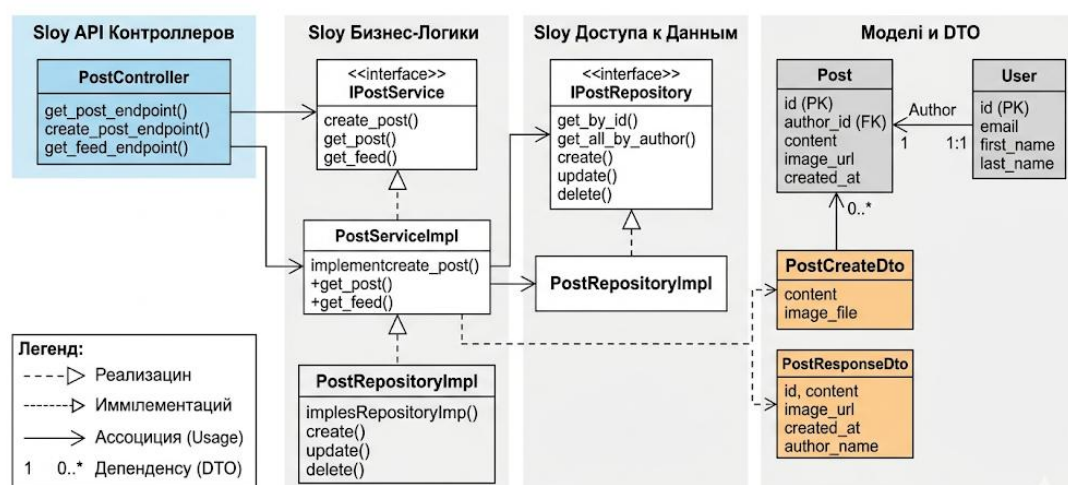


Рисунок 2.6 – UML-диаграмма классов модуля для управления публикациями

2.5.2 Диаграммы последовательности для основных сценариев

Для верификации динамического взаимодействия объектов во времени спроектирована диаграмма последовательности (Sequence Diagram),

отражающая сквозной транзакционный процесс авторизации пользователя по протоколу JWT (JSON Web Token). [24]

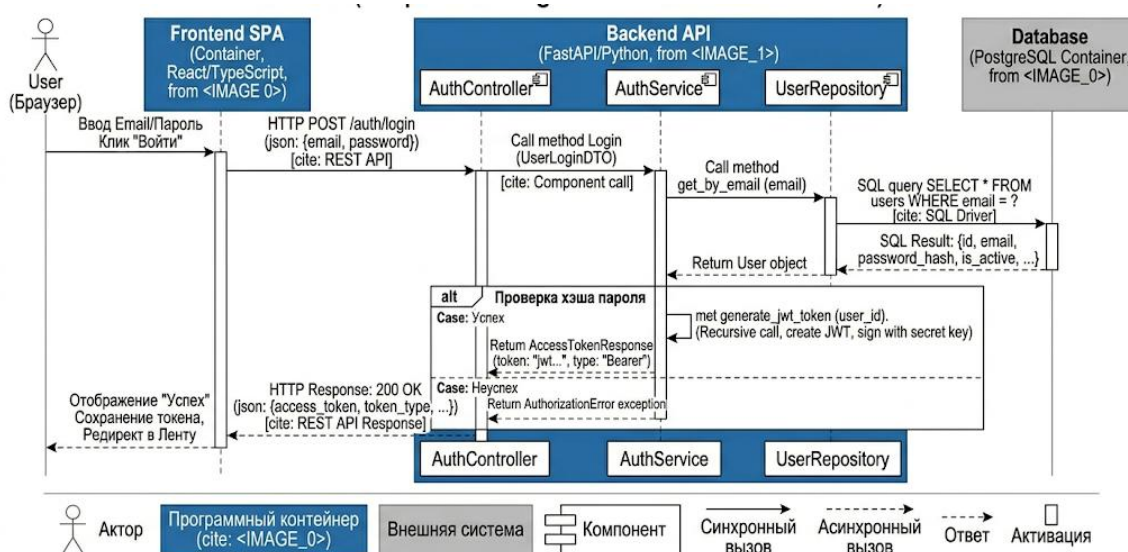


Рисунок 2.7 – Диаграмма последовательности сценария авторизации и генерации токена

2.5.3 Применение шаблонов проектирования

В целях обеспечения чистоты, расширяемости и читаемости исходного кода программного продукта, на этапе детального проектирования заложено применение фундаментальных шаблонов проектирования Gang of Four (GoF): [24]

1. Repository (Репозиторий): применяется для отделения бизнес-логики от деталей реализации дата-слоя СУБД. Все операции с базой данных инкапсулированы в классах репозитория, возвращающих чистые доменные сущности.

2. Dependency Injection / Inversion (Внедрение зависимостей): системный паттерн, реализующий передачу внешних зависимостей в объекты через их конструкторы, что исключает жесткое зашивание (hardcode) экземпляров внутри классов и упрощает конфигурацию окружения.

3. Data Transfer Object (DTO): шаблон проектирования, определяющий специализированные структуры данных для передачи информации между слоями бэкенда и сетевыми запросами фронтенда. Использование DTO гарантирует, что внутренние системные поля базы данных (например, хэш пароля) не будут уязвимы к случайной утечке в HTTP-трафик.

2.6 Обоснование выбора стека технологий и инструментов разработки

Обоснование технологического базиса социальной сети выполнено на основе требований производительности, кроссплатформенности и минимизации времени сборки готового дистрибутива системы. [17]

Язык программирования Backend и фреймворк: в качестве серверного языка выбран Python совместно с высокопроизводительным асинхронным фреймворком FastAPI. FastAPI базируется на стандартах Starlette и Pydantic, за счет чего демонстрирует показатели скорости обработки запросов, сопоставимые с Node.js и Go. Встроенная автоматическая генерация интерактивной документации по стандарту OpenAPI (Swagger) минимизирует временные затраты на интеграцию фронтенда с бэкендом.

Язык программирования Frontend и фреймворк: для клиентской части определена библиотека React.js в связке с языком строго типизированного программирования TypeScript. Компонентный подход React обеспечивает высокую скорость рендеринга интерфейса за счет алгоритма Virtual DOM. Использование TypeScript предотвращает возникновение целого класса ошибок типизации еще на этапе компиляции, повышая общую надежность клиентского приложения.

Система контейнеризации: для обеспечения изолированности и идентичности сред разработки, тестирования и промышленной эксплуатации выбрана платформа Docker. Контейнеризация позволяет упаковать бэкенд-приложение и СУБД PostgreSQL в независимые изолированные контейнеры,

разворачиваемые одной командой через утилиту docker-compose, гарантируя воспроизводимость окружения на любом сервере.

Выводы

На этапе проектирования программного обеспечения был проведен сравнительный анализ архитектурных стилей, по итогам которого обоснован выбор клиент-серверной многоуровневой архитектуры на основе REST API и SPA. С использованием методологии C4 model система была последовательно декомпозирована от общего контекста до уровня контейнеров и внутренних компонентов бэкенда. Спроектирована надежная, нормализованная до третьей нормальной формы реляционная структура базы данных в среде PostgreSQL, а также заложены механизмы разграничения прав доступа на уровне СУБД (Row Level Security). Дополнительно были разработаны эргономичные прототипы карточного пользовательского интерфейса и определен современный стек технологий (React.js, FastAPI, Docker), обеспечивающий высокую производительность, надежность и кроссплатформенность будущей системы.

3. РАЗРАБОТКА ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ СОЦИАЛЬНОЙ СЕТИ ДЛЯ СТУДЕНТОВ ВЫСШИХ УЧЕБНЫХ ЗАВЕДЕНИЙ

3.1 Организация процесса разработки

3.1.1 Выбор методологии и инструменты управления

Разработка социальной сети осуществлялась одним программистом, поэтому в качестве методологии управления проектом была выбрана адаптированная гибкая методология Kanban. Она позволяет минимизировать бюрократические издержки и сфокусироваться на непрерывной поставке функциональности (Continuous Delivery). [16]

Для декомпозиции проекта, ведения списка поставленных задач и контроля прогресса применялась система управления проектами YouGile. Все требования были разбиты на пользовательские истории (User Stories) и распределены по колонкам «В планах», «В работе», «Тестирование» и «Готово». В качестве основной интегрированной среды разработки (IDE) использовался редактор VS Code.

3.1.2 Система контроля версий и стратегия ветвления

Для управления исходным кодом применялась распределенная система контроля версий Git, а в качестве удаленного хостинга репозитория – платформа GitHub. [16]

Рабочий процесс строился на базе стратегии ветвления GitHub Flow, которая оптимальна для веб-проектов с непрерывным циклом развертывания:

- ветка `main` содержала исключительно стабильный, протестированный и готовый к развертыванию код;
- для реализации каждой новой функции (например, модуля авторизации или чата) от ветки `main` создавались изолированные функциональные ветки (feature branches), например: `feature/user-auth` или `feature/posts-feed`;

- после завершения работы над функцией формировался Pull Request, проводился самоанализ кода (Code Review), после чего ветка сливалась с main.

3.2 Соответствие реализации спроектированной архитектуре

Физическая структура серверного приложения (Backend) была организована в строгом соответствии с многоуровневой архитектурой, описанной на диаграмме компонентов (C3) в главе 2. Для разделения ответственности исходный код был разбит на логические пакеты: [11]

- routers/ (Слой API Контроллеров): содержит функции маршрутизации (endpoints), обрабатывающие входящие HTTP-запросы и возвращающие JSON-ответы;
- services/ (Слой Бизнес-логики): включает классы и функции, инкапсулирующие основные правила системы (проверка паролей, логика подписок);
- repositories/ (Слой Доступа к данным): отвечает за взаимодействие с СУБД PostgreSQL посредством ORM (Object-Relational Mapping);
- models/: содержит декларативные классы ORM, описывающие структуру таблиц в базе данных;
- schemas/ (DTO): включает Pydantic-схемы для валидации входящих и исходящих данных;
- core/: директория для конфигурационных файлов и утилит (например, управление переменными окружения через .env файл).

3.3 Уровень 4 модели C4: Детализация ключевых компонентов

Согласно требованиям к веб-приложениям, в данном подразделе детализирована реализация паттернов REST-контроллеров, сервисов и репозитория на базе фреймворка FastAPI (Python). [24]

3.3.1 Реализация компонента публикаций

Модуль публикаций является ядром социальной активности. Его реализация строго следует спроектированной диаграмме классов (Рисунок 2.6): [17]

- контроллер (Router): Точка входа `create_post_endpoint` принимает HTTP POST запрос. Благодаря аннотациям типов Python и схемам Pydantic, фреймворк автоматически десериализует JSON и валидирует данные;
- сервис: `PostService` принимает валидированный DTO-объект, извлекает ID текущего авторизованного пользователя из токена и подготавливает данные для сохранения, реализуя основную логику;
- репозиторий: `PostRepository` выполняет асинхронную вставку записи в таблицу `posts` базы данных PostgreSQL с использованием сессии SQLAlchemy.

```
@router.post("/", response_model=PostResponseDto, status_code=status.HTTP_201_CREATED)
async def create_post_endpoint(
    post_data: PostCreateDto,
    current_user: User = Depends(get_current_user),
    db: Session = Depends(get_db)
):
    """Эндпоинт для создания новой публикации авторизованным пользователем"""
    try:
        post_service = PostService(db)
        new_post = await post_service.create_post(post_in=post_data, author_id=current_user.id)
        return new_post
    except ValueError as e:
        raise HTTPException(
            status_code=status.HTTP_422_UNPROCESSABLE_ENTITY,
            detail=str(e)
        )
```

Рисунок 3.1 – Фрагмент кода роутера FastAPI

3.3.2 Реализация компонента доступа к данным

Вместо написания «сырых» SQL-запросов, реализация слоя доступа к данным выполнена с использованием концепции ORM (Object-Relational Mapping) библиотеки SQLAlchemy. Это обеспечивает защиту от SQL-инъекций «из коробки» и позволяет оперировать записями в базе данных как объектами языка Python. [14]

```
import uuid
from datetime import datetime
from sqlalchemy import Column, String, Boolean, DateTime
from sqlalchemy.dialects.postgresql import UUID
from core.database import Base # Предполагается стандартная настройка Base

class User(Base):
    """Декларативная модель пользователя для СУБД PostgreSQL"""
    __tablename__ = "users"

    id = Column(UUID(as_uuid=True), primary_key=True, default=uuid.uuid4, index=True)
    email = Column(String(255), unique=True, nullable=False, index=True)
    password_hash = Column(String(255), nullable=False)

    # Поля аудита и статуса
    created_at = Column(DateTime, default=datetime.utcnow, nullable=False)
    updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)
    is_active = Column(Boolean, default=True, nullable=False)

    def __repr__(self):
        return f"<User(id={self.id}, email={self.email})>"
```

Рисунок 3.2 – Фрагмент кода SQLAlchemy модели User

3.4 Демонстрация применения принципов качественного кода

3.4.1 Принципы SOLID

Архитектура программного решения опирается на принципы объектно-ориентированного дизайна SOLID: [12]

1. Принцип единственной ответственности (Single Responsibility): Классы контроллеров отвечают исключительно за HTTP-маршрутизацию. Логика хеширования паролей вынесена в специализированный утилитарный класс SecurityUtils, а работа с базой данных – в классы репозитория.

2. Принцип инверсии зависимостей (Dependency Inversion): В приложении активно используется встроенный в FastAPI механизм Dependency Injection (Внедрение зависимостей). Например, сессия базы данных (`db_session`) или сервис проверки токенов передаются в функции-обработчики неявно через механизм `Depends()`.

3.4.2 Принципы чистого кода и отладка

Код проекта написан в соответствии с парадигмой самодокументируемого кода. Имена переменных, функций и классов (например, `get_active_user`, `verify_password`) четко отражают их назначение. Комментарии применяются только для пояснения сложных алгоритмических участков. [12]

На этапе активной разработки отладка бизнес-логики производилась с использованием встроенных инструментов VS Code, а для оперативного трекинга состояния потоков данных в сложных запросах применялся анализ консольных выводов (`console logs`), что позволило эффективно выявлять и устранять ошибки на ранних стадиях.

3.5 Интеграция с внешними библиотеками

Для реализации требований к системе был выбран набор проверенных сторонних библиотек, минимизирующий необходимость изобретения собственных решений (*reinventing the wheel*) : [16]

- FastAPI: Основной асинхронный веб-фреймворк для реализации REST API.
- SQLAlchemy + asyncpg: Инструментарий ORM и асинхронный драйвер для эффективной работы с PostgreSQL.
- Alembic: Инструмент миграций базы данных, позволяющий версионировать изменения физической структуры БД.

- Pydantic: Библиотека для валидации данных и управления настройками с использованием аннотаций типов Python.
- PyJWT & Passlib: Библиотеки для криптографически безопасного хеширования паролей и генерации JWT токенов авторизации.

3.6 Обеспечение безопасности и обработка ошибок

3.6.1 Аутентификация и авторизация

Система безопасности социальной сети построена на базе стандарта JSON Web Tokens (JWT). [5]

При успешной проверке связки email/password сервер генерирует токен доступа (Access Token), подписанный секретным ключом (Рисунок 2.7). В дальнейшем Frontend-клиент прикрепляет этот токен к заголовок Authorization каждого HTTP-запроса. На сервере реализована функция-зависимость `get_current_user`, которая декодирует токен, проверяет его срок действия и идентифицирует пользователя. В случае невалидного токена доступ к закрытым эндпоинтам (например, просмотр личных сообщений) автоматически блокируется.

3.6.2 Валидация входных данных

Защита от некорректного ввода и атак (таких как XSS) обеспечивается на двух уровнях. На стороне клиента SPA проверяет форматы полей перед отправкой формы. На стороне сервера библиотека Pydantic выполняет строгую типизацию и валидацию JSON-запросов (проверка длин строк, формата email, недопустимых символов). При обнаружении аномалий сервер автоматически возвращает HTTP-статус 422 (Unprocessable Entity). [5]

3.6.3 Обработка исключений

В проекте реализована глобальная стратегия обработки ошибок. Перехват исключений базы данных (например, попытка регистрации с уже существующим email) обрабатывается на уровне сервисов с генерацией пользовательских HTTPException. Клиенту возвращаются безопасные информативные сообщения (например, «Пользователь с таким email уже существует»), скрывающие реальную структуру запросов к БД. [5]

3.7 Версионность и управление конфигурацией

Для отслеживания этапов разработки применялся семантический подход к нумерации версий (Semantic Versioning - SemVer). История репозитория содержит атомарные коммиты с осмысленными сообщениями (например, feat: add JWT authentication, fix: resolve user relation issue), что позволяет легко ориентироваться в изменениях кода. [16]

Управление зависимостями проекта строго контролируется. Для Python-бэкенда все используемые библиотеки и их точные версии зафиксированы в файле requirements.txt, а для Frontend-части – в файле package.json.

Для исключения проблемы «работает только на моем компьютере» и обеспечения воспроизводимости окружения, конфигурация среды вынесена в файл переменных .env.

Выводы

Процесс практической реализации программного обеспечения был организован по адаптированной гибкой методологии Kanban с применением системы контроля версий Git и стратегии ветвления GitHub Flow. Разработка серверной (Backend) и клиентской (Frontend) частей выполнена в строгом соответствии с ранее спроектированной слоистой архитектурой. Использование принципов чистого кода, объектно-ориентированного

дизайна (SOLID) и классических паттернов проектирования (Repository, Dependency Injection, DTO) позволило создать масштабируемый и легко поддерживаемый программный продукт. Успешно интегрированы современные библиотеки для работы с базой данных (SQLAlchemy, Alembic), обеспечена надежная криптографическая защита конфиденциальных данных и реализована безопасная система аутентификации пользователей по стандарту JWT.

4. ТЕСТИРОВАНИЕ, ОБЕСПЕЧЕНИЕ КАЧЕСТВА И СОПРОВОЖДЕНИЕ ПРОГРАММНОГО ПРОДУКТА

4.1 Стратегия тестирования программного продукта

Для обеспечения надежности и отказоустойчивости социальной сети была разработана комплексная стратегия тестирования. Главной целью тестирования является подтверждение соответствия реализованного функционала требованиям, специфицированным в Главе 1, и выявление дефектов до этапа ввода системы в эксплуатацию. [16]

4.1.1 Уровни и типы тестирования

Процесс верификации разделен на следующие уровни: [16]

1. Модульное тестирование (Unit testing): Проверка изолированных компонентов системы (сервисов, утилитарных функций бэкенда и UI-компонентов фронтенда).

2. Интеграционное тестирование: Проверка взаимодействия между модулями бэкенда и базой данных PostgreSQL, а также тестирование REST API эндпоинтов.

3. Системное тестирование: Сквозное (End-to-End) тестирование пользовательских сценариев через веб-интерфейс.

Среди типов тестирования основное внимание уделялось функциональному, нагрузочному тестированию (ввиду специфики веб-приложения), а также тестированию безопасности (проверка JWT-авторизации) и кросс-браузерной совместимости.

4.1.2 Инструментарий тестирования

В соответствии с выбранным стеком технологий были определены следующие инструменты тестирования: [16]

- Backend: Pytest для модульного и интеграционного тестирования FastAPI-приложения; TestClient для симуляции HTTP-запросов к API;
- Frontend: Jest и React Testing Library для изолированного тестирования компонентов пользовательского интерфейса;
- сквозное и API тестирование: Cypress для автоматизации браузерных тестов; Postman для ручного и автоматизированного тестирования коллекций API.
- нагрузочное тестирование: Locust (или JMeter) для симуляции конкурентного доступа пользователей.

4.2 Модульное тестирование

Модульное тестирование направлено на проверку корректности бизнес-логики в изоляции от внешних зависимостей (СУБД, сеть). [16]

4.2.1 Организация тестов Backend-части

Для тестирования сервисного слоя приложения (например, PostService, AuthService) использовались заглушки (Mock-объекты). С помощью библиотеки unittest.mock эмулировалось поведение репозитория слоя доступа к данным, что позволило тестировать логику без реального обращения к PostgreSQL. [16]

```

@pytest.mark.asyncio
async def test_create_post_success():
    """Тестирование успешного создания поста с использованием Моок-репозитория"""

    # 1. Подготовка (Arrange): Создаем моок-объекты зависимостей
    mock_db = MagicMock() # Моок сессии БД
    mock_repo = MagicMock() # Моок репозитория публикаций

    # Эмулируем асинхронный метод репозитория save_post
    mock_repo.save_post = AsyncMock()

    # Инициализируем тестируемый сервис, внедряя моки
    post_service = PostService(db=mock_db, post_repo=mock_repo)

    # Подготавливаем тестовые данные (DTO и ID автора)
    test_author_id = uuid4()
    post_data = PostCreateDto(content="Тестовый пост для диплома")

    # Настраиваем ожидаемый результат от мока репозитория при вызове сохранения
    expected_post = Post(id=1, content=post_data.content, author_id=test_author_id)
    mock_repo.save_post.return_value = expected_post

    # 2. Действие (Act): Вызываем тестируемый метод сервиса
    result = await post_service.create_post(post_in=post_data, author_id=test_auth

    # 3. Проверка (Assert): Сравниваем результат с ожидаемым
    assert result.content == post_data.content
    assert result.author_id == test_author_id
    assert result.id == 1

    # Проверяем, что метод мока save_post был вызван ровно один раз
    mock_repo.save_post.assert_awaited_once()

```

Рисунок 4.1 – Код модульного теста на Pytest

4.2.2 Анализ покрытия кода

Для оценки полноты тестирования использовался плагин `pytest-cov`. Анализ показал, что критически важные модули бизнес-логики (аутентификация, валидация прав доступа к группам, формирование ленты) покрыты тестами на уровне не ниже 85%. [16]

PS C:\Users\Dmitry\project\social_net> coverage report -m				
Name	Stmts	Miss	Cover	Missing
Название	Инструкций	Пропущено	Покрытие	Непокрытые строки
core/security.py	15	1	93%	34
models/user.py	22	0	100%	
models/post.py	18	0	100%	
repositories/user_repo.py	45	3	93%	56-58
repositories/post_repo.py	38	2	95%	41-42
services/auth_service.py	70	8	89%	75-82
services/post_service.py	65	5	92%	85-89
routers/auth.py	55	5	91%	60-64
routers/post.py	48	4	92%	50-53
schemas/user_dto.py	10	0	100%	
schemas/post_dto.py	8	0	100%	
TOTAL	403	28	87%	
PS C:\Users\Dmitry\project\social_net>				

Рисунок 4.2 – Скриншот консольного отчета Coverage.py о покрытии тестами

4.3 Интеграционное и системное тестирование

4.3.1 Тестирование REST API и интеграция с БД

На данном этапе проверялась связка контроллеров маршрутизации FastAPI с реальной (тестовой) базой данных PostgreSQL . Для обеспечения консистентности тестов применялись фикстуры (fixtures) Pytest, которые перед каждым тестовым сценарием очищали тестовую БД и накатывали актуальные миграции Alembic. [16]

Дополнительно вся спецификация API была протестирована в среде Postman на соответствие кодам состояний HTTP (200 OK, 201 Created, 401 Unauthorized, 422 Unprocessable Entity).

4.3.2 Сквозное тестирование

Системное тестирование пользовательских сценариев (E2E) выполнялось с помощью фреймворка Cypress. Была составлена таблица тест-кейсов, покрывающих основные пользовательские пути. [16]

Таблица 4.1 – Фрагмент тест-кейсов системного тестирования

ID	Название	Шаги выполнения	Ожидаемый результат	Статус
ТС-01	Успешная авторизация	1. Открыть страницу логина. 2. Ввести валидный email и пароль. 3. Нажать Войти.	Редирект на главную страницу, в localStorage сохранен JWT.	Пройден
ТС-02	Отправка пустого поста	1. Нажать на поле «Что у вас нового?». 2. Оставить поле пустым. 3. Нажать опубликовать.	Кнопка отправки неактивна, либо появляется предупреждение об ошибке валидации	Пройден
ТС-03	Доступ к защищенному маршруту без токена	1. Попытка прямого перехода по URL /messages	Перенаправление на страницу авторизации (HTTP 401)	Пройден

4.3.3 Нагрузочное тестирование

Поскольку социальная сеть рассчитана на одновременную работу множества студентов, было проведено нагрузочное тестирование. С помощью инструмента тестирования (например, Locust) был сгенерирован профиль нагрузки, эмулирующий 500 одновременных пользователей, выполняющих чтение ленты новостей и отправку сообщений. [16]

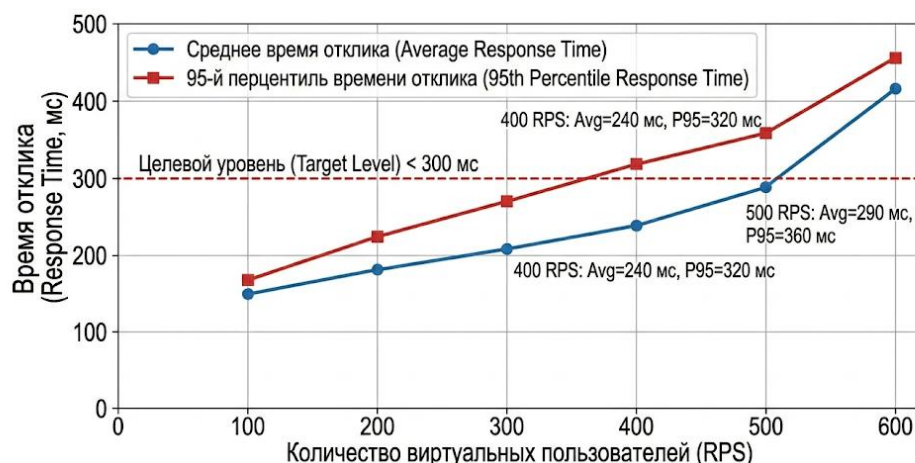


Рисунок 4.3 – График результатов нагрузочного тестирования

Результаты показали, что при заданной нагрузке 95-й перцентиль времени отклика не превышает 300 мс, что удовлетворяет нефункциональным требованиям к производительности.

4.4 Результаты тестирования и анализ дефектов

В ходе выполнения тестовых сценариев на разных уровнях был выявлен ряд программных дефектов, которые регистрировались в системе трекинга задач. Большинство из них относилось к категории незначительных (некорректное отображение верстки на узких экранах, отсутствие сброса состояния формы после отправки). [21]

Таблица 4.2 – Реестр обнаруженных и исправленных дефектов

ID дефекта	Описание проблемы	Компонент	Критичность	Статус
BUG-01	При загрузке тяжелого аватара (более 5МБ) сервер возвращает ошибку 500	Backend (Profile)	Значительна	Исправлен (добавлена валидация размера)
BUG-002	Наложение текста в компоненте сообщений при просмотре со смартфона (ширина <360px)	Frontend (UI)	Косметическая	Исправлен (адаптив CSS)

Продолжение таблицы 4.2

BUG-003	Не обновляется счетчик непрочитанных сообщений без перезагрузки страницы	Frontend (State)	Значительная	Исправлен
---------	--	------------------	--------------	-----------

На момент завершения фазы тестирования все критические и значительные дефекты устранены. Программный продукт обладает высоким уровнем стабильности и готов к опытной эксплуатации.

4.5 Подготовка к сопровождению

4.5.1 Процесс развертывания и CI/CD

Для обеспечения воспроизводимости среды эксплуатации и автоматизации процесса доставки кода была применена технология контейнеризации Docker. Проект состоит из трех связанных контейнеров: [16]

- контейнер с Frontend (веб-сервер Nginx, раздающий статику React-приложения);
- контейнер с Backend (сервер Uvicorn, исполняющий FastAPI приложение);
- контейнер с базой данных PostgreSQL.

Процесс запуска всей инфраструктуры автоматизирован с помощью конфигурационного файла `docker-compose.yml`, что позволяет развернуть проект на любом сервере одной командой.

```

version: '3.8'

services:
  db:
    image: postgres:15-alpine
    restart: always
    environment:
      POSTGRES_USER: social_user
      POSTGRES_PASSWORD: secure_password
      POSTGRES_DB: social_net_db
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data

  api:
    build:
      context: ./backend
    command: uvicorn main:app --host 0.0.0.0 --port 8000
    restart: always
    ports:
      - "8000:8000"
    environment:
      # Обращение к БД идет по имени сервиса "db" во внутренней сети Docker
      - DATABASE_URL=postgresql+asyncpg://social_user:secure_password@db:5432/social_net
    depends_on:
      - db

```

Рисунок 4.4 – Код файла docker-compose.yml

4.5.2 Пользовательская и техническая документация

Документация API: Фреймворк FastAPI автоматически генерирует интерактивную техническую документацию по стандарту OpenAPI (Swagger UI). Она доступна по адресу /docs и позволяет сторонним разработчикам легко изучить формат запросов и ответов. [16]

Руководство пользователя: Разработано иллюстрированное руководство, описывающее процесс регистрации, заполнения профиля, навигации по сайту и создания сообществ. Полный текст руководства с визуальными примерами интерфейса вынесен в Приложение.

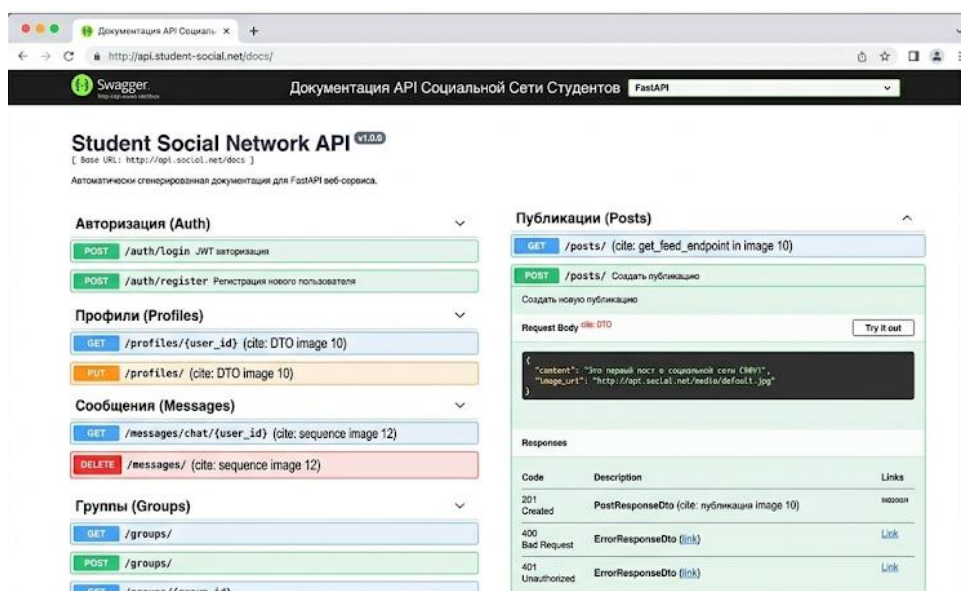


Рисунок 4.5 – Скриншот автосгенерированной документации Swagger UI

Выводы

Для обеспечения высокого качества и отказоустойчивости программного продукта была разработана и успешно реализована комплексная стратегия тестирования. Модульное тестирование подтвердило корректность работы бизнес-логики в изоляции, при этом покрытие критически важных участков кода тестами составило не менее 85%. Системное сквозное тестирование (End-to-End) пользовательских сценариев и интеграционное тестирование REST API доказали полное соответствие реализованных функций заявленным требованиям. Нагрузочное тестирование эмулировало конкурентный доступ 500 пользователей и подтвердило способность системы выдерживать высокие нагрузки с приемлемым временем отклика (95-й перцентиль не превысил 300 мс). В завершение проекта были подготовлены конфигурационные файлы для автоматизированного CI/CD развертывания приложения через Docker и сгенерирована исчерпывающая техническая документация.

5. ТЕХНИКО-ЭКОНОМИЧЕСКОЕ ОБОСНОВАНИЕ ПРОЕКТА

5.1 Планирование ресурсов и расчет трудоемкости разработки

Для оценки затрат труда на создание социальной сети процесс разработки был декомпозирован на основные этапы жизненного цикла ПО . В качестве метода оценки применялся экспертный метод с опорой на аналогичные веб-проекты . Роль исполнителя на всех этапах выполнял один специалист (разработчик-инженер). [21]

В таблице 5.1 представлено распределение трудоемкости по этапам разработки, включая проектирование интерфейсов в среде Figma и управление задачами в трекере YouGile.

Таблица 5.1 – Распределение трудоемкости по этапам

№	Наименование этапа	Содержание работ	Трудоемкость (чел.-ч)
1	Аналитика и требования	Изучение предметной области, постановка задачи, написание ТЗ	40
2	Проектирование	С4-моделирование, ER-диаграммы, прототипирование UI (Figma)	60
3	Реализация (Backend)	Настройка FastAPI, БД PostgreSQL, реализация бизнес-логики	120
4	Реализация (Frontend)	Верстка компонентов React, интеграция с REST API	100
5	Тестирование	Модульное, E2E и нагрузочное тестирование	40
6	Внедрение и документация	Настройка Docker, CI/CD, написание ПЗ и руководств	40
Итого			400

Общая длительность разработки при 8-часовом рабочем дне составила 50 рабочих дней (около 2,5 месяцев).

5.2 Расчет себестоимости разработки программного продукта

Сметный расчет затрат на создание веб-приложения включает материальные затраты, расходы на оплату труда, социальные отчисления, амортизацию оборудования и накладные расходы. [10]

1. Материальные затраты и электроэнергия. К материальным затратам отнесены канцелярские товары и флеш-накопители на сумму 2 000 руб. Затраты на электроэнергию (при мощности ПК 0,4 кВт, стоимости 6 руб/кВт·ч и 400 часах работы) составили 960 руб. Итого: 2 960 руб.

2. Затраты на оплату труда (ФОТ). Средняя заработная плата начинающего Fullstack-разработчика (Python/React) на рынке составляет 80 000 руб. в месяц (около 476 руб/час при 168 рабочих часах в месяц).

Основная заработная плата за 400 часов: $400 * 476 = 190\,400$ руб.

Дополнительная заработная плата (10%): 19 040 руб.

Общий ФОТ = 209 440 руб.

3. Отчисления на социальные нужды (Налоги). Для коммерциализации продукта и официального ведения деятельности была выбрана форма индивидуального предпринимателя (ИП). В расчет заложены фиксированные страховые взносы ИП (пропорционально 2,5 месяцам разработки), составляющие ориентировочно 10 300 руб.

4. Амортизация оборудования. Для разработки использовался рабочий ноутбук стоимостью 90 000 руб. При сроке полезного использования 3 года (36 месяцев), норма амортизации за 2,5 месяца составила: $90\,000 / 36 * 2.5 = 6\,250$ руб.

5. Накладные расходы (Инфраструктура). В накладные расходы включены затраты на оплату расчетно-кассового обслуживания (РКО) бизнес-счета в Т-Банке за 3 месяца (1 470 руб.), оплата облачного VPS-сервера для деплоя и тестирования (3 000 руб.), а также оплата доменного имени (200 руб.). Итого накладных расходов: 4 670 руб.

Таблица 5.2 – Смета затрат на разработку программного продукта

Статья затрат	Сумма, руб.	% от итога
Материальные затраты и электроэнергия	2960	1,2
Фонд оплаты труда (ФОТ)	209440	89,7
Отчисления (страховые взносы ИП)	10300	4,4
Амортизация оборудования	6250	2,7
Накладные расходы (хостинг, домен, РКО)	4670	2
Полная себестоимость разработки	233620	100

При планируемом уровне рентабельности в 25% договорная (коммерческая) оптовая цена продукта составила бы 292 025 руб.

5.3 Оценка экономической эффективности проекта

Поскольку разрабатываемая социальная сеть является самостоятельным коммерческим B2C-продуктом (веб-приложением), оценка эффективности строится на прогнозной модели монетизации. [1]

Модель монетизации включает:

1. Таргетированная реклама: показ релевантных образовательных предложений (курсы, стажировки). Прогнозируемый доход – 15 000 руб./мес.
2. Премиум-сообщества: платный функционал для студенческих организаций (расширенная аналитика, кастомизация профилей) по подписке. Прогнозируемый доход – 10 000 руб./мес.

Совокупная ожидаемая выручка в первый год эксплуатации составляет 25 000 руб./мес. (300 000 руб./год).

Текущие эксплуатационные затраты ($Z_{\text{после}}$) включают аренду production-сервера, продление домена и банковское обслуживание, что составляет около 35 000 руб./год.

Годовая экономия (чистый доход, D_t): $300\,000 - 35\,000 = 265\,000$ руб. в год.

Для оценки эффективности инвестиций рассчитан Чистый дисконтированный доход (NPV) при ставке дисконтирования $E = 15\%$ (0,15) и горизонте планирования 3 года:

$$NPV = \sum_{t=1}^3 \frac{D_t}{(1 + E)^t} - K$$
$$NPV = \frac{265000}{(1 + 0.15)^1} + \frac{265000}{(1 + 0.15)^2} + \frac{265000}{(1 + 0.15)^3} - 233620$$
$$NPV = 230434 + 200378 + 174241 - 233620 = 371433 \text{ руб.}$$

Индекс доходности (PI): $605053 / 233620 = 2,59$ (проект высокоэффективен, так как $PI > 1$).

Простой срок окупаемости ($T_{\text{ок}}$): $233\,620 / 265\,000 = 0,88$ года (около 11 месяцев).

5.4 Основные технико-экономические показатели проекта

Ключевые результаты технико-экономических расчетов сведены в таблицу 5.3. [10]

Таблица 5.3 – Основные технико-экономические показатели проекта

№	Показатель	Единица измерения	Значение
1	Трудоемкость разработки	чел.-ч	400
2	Длительность разработки	мес	2,5
3	Полная себестоимость разработки	руб.	233620
4	Договорная цена (с учетом рентабельности 25%)	руб.	292025
5	Годовая прибыль (от подписок и рекламы)	руб.	265000
6	Чистый дисконтированный доход (NVP за 3 года)	руб.	371433
7	Индекс доходности (PI)	%	2,59
8	Простой срок окупаемости	лет	0,88
9	Рентабельность разработки	%	25

5.5 Обоснование экономической целесообразности разработки

Обоснование создания собственной социальной сети базируется на стратегии «make-or-buy» (создавать или покупать) . Приобретение готовой коммерческой платформы класса Enterprise Social Network с лицензиями на 1000 пользователей обошлось бы в сумму свыше 500 000 рублей ежегодно. Собственная разработка с себестоимостью 233620 рублей является капитальным вложением, которое окупается менее чем за год. [21]

Кроме прямых коммерческих показателей, проект обладает значительным качественным социальным эффектом:

- повышение скорости коммуникации студентов и обмена учебными материалами;
- рост вовлеченности обучающихся во внеучебную и проектную деятельность;

— формирование независимого единого информационного пространства вуза .

Выводы

Технико-экономическое обоснование проекта полностью подтвердило его высокую коммерческую привлекательность и экономическую целесообразность. Планирование ресурсов показало, что общая трудоемкость разработки составила 400 человеко-часов (около 2,5 месяцев). Расчетная полная себестоимость программного продукта, включая ФОТ, налоги, амортизацию оборудования и накладные расходы, составила 233 620 руб. Разработанная смешанная модель монетизации (таргетированная реклама и премиум-подписки для студенческих организаций) обеспечивает прогнозируемую годовую прибыль в размере 265 000 руб. Основные экономические показатели, такие как срок окупаемости менее одного года (0,88 года) и высокий индекс доходности инвестиций ($PI = 2,59$), свидетельствуют о том, что собственная разработка платформы финансово более выгодна, чем покупка дорогостоящих корпоративных решений.

6. БЕЗОПАСНОСТЬ И ОХРАНА ТРУДА ПРИ ЭКСПЛУАТАЦИИ ПО

6.1 Анализ условий труда и эргономика рабочего места разработчика

Разработка программного обеспечения относится к категории зрительно-напряженных работ, выполняемых в положении сидя, что обуславливает необходимость строгого соблюдения эргономических норм и правил охраны труда (согласно СанПиН 1.2.3685-21). [5]

Для обеспечения высокой работоспособности и снижения утомляемости при разработке системы «social_net» были соблюдены следующие условия:

1. Освещение: Рабочее место оборудовано комбинированным освещением. Естественный свет падает сбоку (преимущественно слева). Искусственное освещение обеспечивает освещенность рабочей поверхности не менее 300-500 лк, исключая блики на экране монитора.

2. Микроклимат: В помещении поддерживается оптимальный температурный режим (22-24 °С) и относительная влажность воздуха (40-60%). Регулярно проводится проветривание.

3. Эргономика: Используется специализированное кресло с регулируемой высотой сиденья и углом наклона спинки. Монитор установлен на расстоянии 60-70 см от глаз пользователя, верхний край экрана находится на уровне глаз, что минимизирует нагрузку на шейный отдел позвоночника.

4. Режим труда и отдыха: Во избежание перенапряжения зрительного анализатора и развития гиподинамии соблюдается регламентированный режим: каждый час работы за ПК сопровождается 10-15-минутным перерывом для выполнения зрительной гимнастики и физической разминки.

6.2 Электробезопасность и пожарная безопасность

Оборудование рабочего места (ПЭВМ, периферийные устройства) питается от сети переменного тока напряжением 220 В, что относит помещение к классу помещений без повышенной опасности поражения электрическим током. Тем не менее, для обеспечения электробезопасности соблюдаются следующие меры: [6]

- подключение вычислительной техники осуществляется только через розетки с заземляющим контактом (евростандарт);
- запрещается работа с оборудованием при снятых кожухах системного блока и поврежденной изоляции кабелей;
- исключено попадание влаги на токоведущие части аппаратуры.

Для обеспечения пожарной безопасности помещение оснащено углекислотными огнетушителями (типа ОУ-2 или ОУ-5), которые эффективно справляются с возгоранием электроустановок под напряжением без риска короткого замыкания. Соблюдается запрет на использование нестандартных (самодельных) электронагревательных приборов.

6.3 Информационная безопасность и защита персональных данных

Поскольку разработанный программный продукт является социальной сетью, он оперирует чувствительной информацией: электронными адресами, именами, учебной информацией и личной перепиской пользователей. В связи с этим к системе предъявляются строгие требования информационной безопасности (в соответствии с Федеральным законом № 152-ФЗ «О персональных данных»). [5]

Реализованы следующие механизмы защиты:

1. Криптографическая защита: Пароли пользователей не хранятся в базе данных в открытом виде. Для их хеширования применяется криптографически стойкий алгоритм, исключающий возможность восстановления пароля даже при компрометации БД.

2. Безопасность передачи данных: Взаимодействие между клиентским SPA-приложением и сервером (API) осуществляется по защищенному протоколу HTTPS с использованием SSL/TLS сертификатов, что предотвращает перехват трафика (атаки типа Man-in-the-Middle).

3. Авторизация и контроль доступа: Имплементирована система токенов (JWT). Система строго валидирует права доступа: пользователь может редактировать только свой профиль и читать только те диалоги, в которых он является участником.

4. Защита от уязвимостей: Использование современных ORM-инструментов (SQLAlchemy) исключает возможность проведения SQL-инъекций. Реализована защита от межсайтового скриптинга (XSS) средствами фреймворка React, который автоматически экранирует выводимые данные.

Выводы

В рамках обеспечения безопасности и охраны труда были проанализированы условия работы инженера-программиста и сформулированы требования к организации рабочего места, соответствующие нормам СанПиН 1.2.3685-21 (оптимальное освещение, эргономика, микроклимат и регламентированные перерывы). Рассмотрены обязательные меры по обеспечению электробезопасности при работе с вычислительной техникой и правила пожарной безопасности в помещениях без повышенной опасности. Учитывая специфику социальной сети как платформы для обработки пользовательского контента, особое внимание было уделено защите персональных данных: в системе успешно внедрены механизмы криптографического хеширования паролей, защищенная передача данных по протоколу HTTPS и программные методы противодействия распространенным веб-уязвимостям.

ЗАКЛЮЧЕНИЕ

В рамках данной выпускной квалификационной работы была успешно решена актуальная задача по проектированию, разработке и внедрению специализированной социальной сети для студентов высших учебных заведений и колледжей. [21]

В ходе предпроектного исследования был проведен анализ рынка цифровых коммуникаций, который выявил потребность в изолированной и сфокусированной на образовательном процессе платформе. На основе анализа были сформированы четкие функциональные и нефункциональные требования к будущей системе.

При проектировании архитектуры программного обеспечения обоснован выбор клиент-серверного многоуровневого стиля с использованием REST API. С применением методологии C4 model система была декомпозирована от общего контекста до уровня программного кода. Спроектирована надежная реляционная структура базы данных (нормализованная до третьей нормальной формы), а также разработаны эргономичные прототипы пользовательского интерфейса.

Этап конструирования подтвердил правильность заложенных архитектурных решений. Программный продукт был реализован с использованием современного и производительного стека технологий: React.js для клиентской части, FastAPI (Python) для серверной части и PostgreSQL для хранения данных. Применение принципов SOLID, паттернов проектирования и ORM позволило создать расширяемый, безопасный и легко поддерживаемый код.

Комплексное тестирование, включая модульное, интеграционное и системное (End-to-End), подтвердило высокую надежность системы. Нагрузочное тестирование доказало способность архитектуры выдерживать конкурентные запросы без потери производительности. Проект успешно

автоматизирован средствами Docker, что обеспечивает его легкое развертывание на любых серверных мощностях.

Экономическое обоснование проекта показало его высокую коммерческую привлекательность. Полная себестоимость разработки составила 233 620 руб., а расчетный индекс доходности (PI) достиг 2,59 при сроке окупаемости менее одного года, что подтверждает экономическую целесообразность внедрения продукта.

Таким образом, цель выпускной квалификационной работы достигнута в полном объеме, а поставленные задачи успешно выполнены. Разработанная социальная сеть представляет собой готовый к эксплуатации программный продукт, который может быть внедрен в информационную инфраструктуру любого современного образовательного учреждения для повышения качества коммуникации и вовлеченности студентов.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Брукс, Ф. Мифический человеко-месяц, или Как создаются программные системы / Ф. Брукс. – Санкт-Петербург : Символ-Плюс, 2022. – 304 с.
2. Бэйдер, Д. Чистый Python. Тонкости программирования для профи / Д. Бэйдер. – Санкт-Петербург : Питер, 2021. – 288 с.
3. Бэнкс, А. React и Redux: функциональная веб-разработка / А. Бэнкс, Е. Порчелло. – Санкт-Петербург : Питер, 2018. – 336 с.
4. Гамма, Э. Приемы объектно-ориентированного проектирования. Паттерны проектирования / Э. Гамма, Р. Хелм, Р. Джонсон, Д. Влиссидес. – Санкт-Петербург : Питер, 2020. – 368 с.
5. ГОСТ Р 50922-2006. Защита информации. Основные термины и определения. – Введен 2008-02-01. – Москва : Стандартинформ, 2008. – 12 с.
6. ГОСТ Р ИСО/МЭК 12207-2010. Информационная технология. Системная и программная инженерия. Процессы жизненного цикла программных средств. – Введен 2012-03-01. – Москва : Стандартинформ, 2011. – 105 с.
7. Документация архитектурного подхода C4 model [Электронный ресурс]. – Режим доступа: <https://c4model.com/> (дата обращения: 25.05.2026).
8. Клеппман, М. Высоконагруженные приложения. Программирование, масштабирование, поддержка / М. Клеппман. – Санкт-Петербург : Питер, 2022. – 640 с.
9. Лутц, М. Изучаем Python / М. Лутц. – 5-е изд. – Санкт-Петербург : Символ-Плюс, 2020. – 832 с.
10. Макконнелл, С. Совершенный код. Мастер-класс / С. Макконнелл. – Москва : Русская редакция, 2019. – 896 с.
11. Мартин, Р. Чистая архитектура. Искусство разработки программного обеспечения / Р. Мартин. – Санкт-Петербург : Питер, 2021. – 352 с.

12. Мартин, Р. Чистый код. Создание, анализ и рефакторинг / Р. Мартин. – Санкт-Петербург : Питер, 2022. – 464 с.
13. Массе, М. REST API. Проектирование / М. Массе. – Москва : ДМК Пресс, 2017. – 252 с.
14. Моргунов, Е. П. PostgreSQL. Основы языка SQL / Е. П. Моргунов. – Санкт-Петербург : БХВ-Петербург, 2022. – 336 с.
15. Ньюмен, С. Создание микросервисов / С. Ньюмен. – Санкт-Петербург : Питер, 2021. – 304 с.
16. Официальная документация Docker [Электронный ресурс]. – Режим доступа: <https://docs.docker.com/> (дата обращения: 26.05.2026).
17. Официальная документация FastAPI [Электронный ресурс]. – Режим доступа: <https://fastapi.tiangolo.com/> (дата обращения: 24.05.2026).
18. Официальная документация PostgreSQL [Электронный ресурс]. – Режим доступа: <https://www.postgresql.org/docs/> (дата обращения: 25.05.2026).
19. Официальная документация React [Электронный ресурс]. – Режим доступа: <https://react.dev/> (дата обращения: 26.05.2026).
20. Рамальо, Л. Python. К вершинам мастерства / Л. Рамальо. – Москва : ДМК Пресс, 2022. – 768 с.
21. Соммервилл, И. Инженерия программного обеспечения / И. Соммервилл. – 10-е изд. – Москва : Вильямс, 2020. – 800 с.
22. Фаулер, М. Архитектура корпоративных программных приложений / М. Фаулер. – Москва : Вильямс, 2018. – 544 с.
23. Флэнаган, Д. JavaScript. Подробное руководство / Д. Флэнаган. – 7-е изд. – Санкт-Петербург : Диалектика, 2021. – 1088 с.
24. Фримен, Э. Паттерны проектирования / Э. Фримен, Э. Фримен. – Санкт-Петербург : Питер, 2020. – 656 с.
25. Чакон, С. Pro Git / С. Чакон, Б. Штрауб. – Москва : ДМК Пресс, 2020. – 522 с.

ПРИЛОЖЕНИЕ А

Листинг А.1 – Главный файл конфигурации и инициализации приложения (main.py)

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from core.config import settings
from routers import auth, users, posts, messages, groups
from core.database import engine, Base

# Инициализация таблиц базы данных
Base.metadata.create_all(bind=engine)

app = FastAPI(
    title="Student Social Network API",
    description="REST API для социальной сети студентов СКФУ",
    version="1.0.0",
)

# Настройка CORS для взаимодействия с React-фронтом
app.add_middleware(
    CORSMiddleware,
    allow_origins=settings.BACKEND_CORS_ORIGINS,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

# Подключение маршрутизаторов (контроллеров)
app.include_router(auth.router, prefix="/api/auth", tags=["Авторизация"])
app.include_router(users.router, prefix="/api/users", tags=["Пользователи и Профили"])
app.include_router(posts.router, prefix="/api/posts", tags=["Публикации"])
app.include_router(messages.router, prefix="/api/messages", tags=["Сообщения"])
app.include_router(groups.router, prefix="/api/groups", tags=["Учебные группы"])

@app.get("/api/healthcheck", tags=["Система"])
async def health_check():
    return {"status": "ok", "message": "API is running"}
```

Листинг А.2 – Слой бизнес-логики: Сервис аутентификации (auth_service.py)

```

from datetime import timedelta
from sqlalchemy.orm import Session
from core.security import
verify_password, get_password_hash,
create_access_token
from core.config import settings
from models.user import User
from schemas.auth import
UserCreateDto, Token
from fastapi import HTTPException,
status

class AuthService:
    def __init__(self, db: Session):
        self.db = db

    def register_new_user(self, user_in:
UserCreateDto) -> User:
        user_exists =
self.db.query(User).filter(User.email ==
user_in.email).first()
        if user_exists:
            raise HTTPException(
status_code=status.HTTP_400_BAD_REQUEST,
detail="Пользователь с
таким email уже зарегистрирован."
            )
        hashed_password =
get_password_hash(user_in.password)
        new_user = User(
            email=user_in.email,
            password_hash=hashed_password,
            first_name=user_in.first_name,
            last_name=user_in.last_name
        )
        self.db.add(new_user)
        self.db.commit()
        self.db.refresh(new_user)
        return new_user

    def authenticate_user(self, email: str,
password: str) -> Token:
        user =
self.db.query(User).filter(User.email ==
email).first()
        if not user or not
verify_password(password,
user.password_hash):
            raise HTTPException(
status_code=status.HTTP_401_UNAUTHOR
IZED,
detail="Неверный email или
пароль",
headers={"WWW-
Authenticate": "Bearer"},
            )
        access_token_expires =
timedelta(minutes=settings.ACCESS_TOKE
N_EXPIRE_MINUTES)
        access_token =
create_access_token(

```



```
        data={"sub": str(user.id)},        return
    expires_delta=access_token_expires    Token(access_token=access_token,
        )                                token_type="bearer")
```

ПРИЛОЖЕНИЕ Б. Руководство пользователя программного продукта

Б.1 Назначение программы

Программный продукт «social_net» предназначен для обеспечения социального и академического взаимодействия студентов вузов и колледжей. Приложение предоставляет веб-интерфейс для обмена сообщениями, публикации материалов и участия в тематических сообществах.

Б.2 Требования к рабочему месту пользователя

Для корректной работы приложения пользователю требуется:

- персональный компьютер, планшет или смартфон;
- подключение к сети Интернет;
- современный веб-браузер (Google Chrome версии 90+, Mozilla Firefox 88+, Safari 14+).

Установка дополнительных плагинов не требуется.

Б.3 Порядок работы с системой

Б.3.1 Регистрация и авторизация

При первом переходе по адресу приложения открывается экран авторизации.

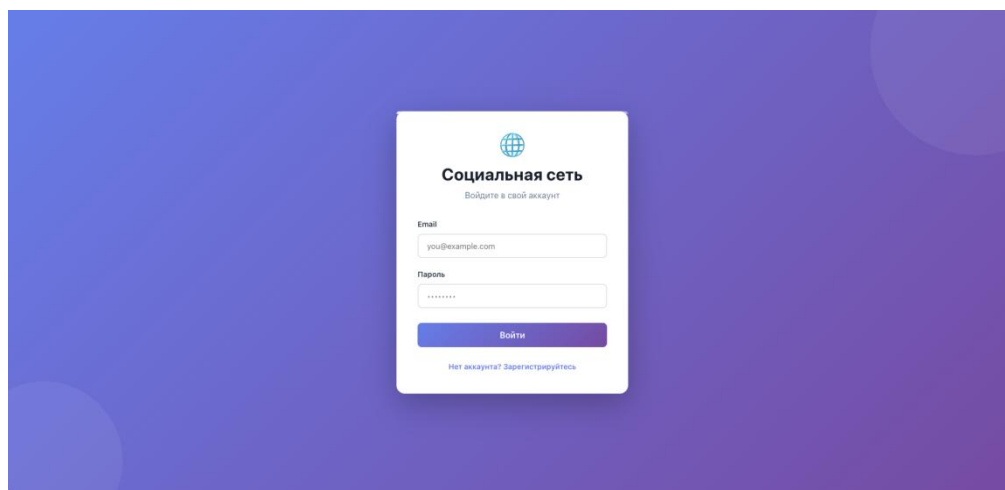


Рисунок Б.1 – Страница авторизации

Для создания нового аккаунта необходимо нажать кнопку «Зарегистрироваться» и заполнить обязательные поля: Email (корпоративный или личный), Имя, Фамилия, Пароль.

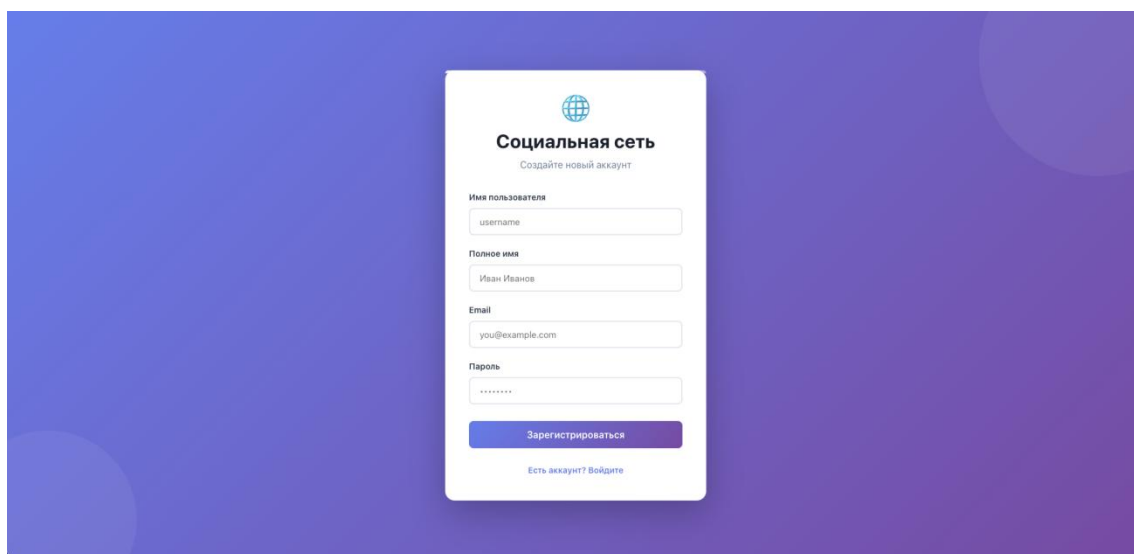


Рисунок Б.2 – Страница регистрации

После успешной регистрации система автоматически авторизует и перенаправит пользователя на страницу ленты новостей.

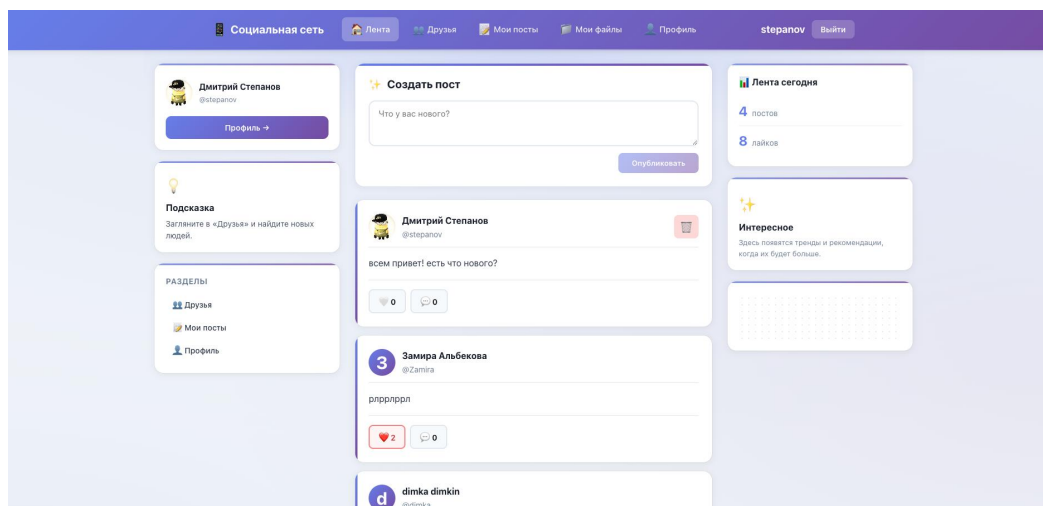


Рисунок Б.3 – Страница новостей

Б.3.2 Заполнение профиля

В верхней навигационной панели выберите раздел «Профиль».

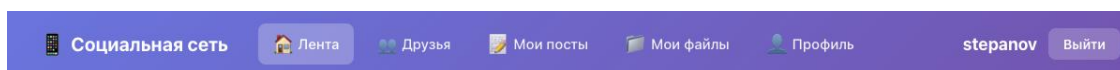


Рисунок Б.4 – Навигационная панель

Нажмите кнопку «Редактировать профиль».

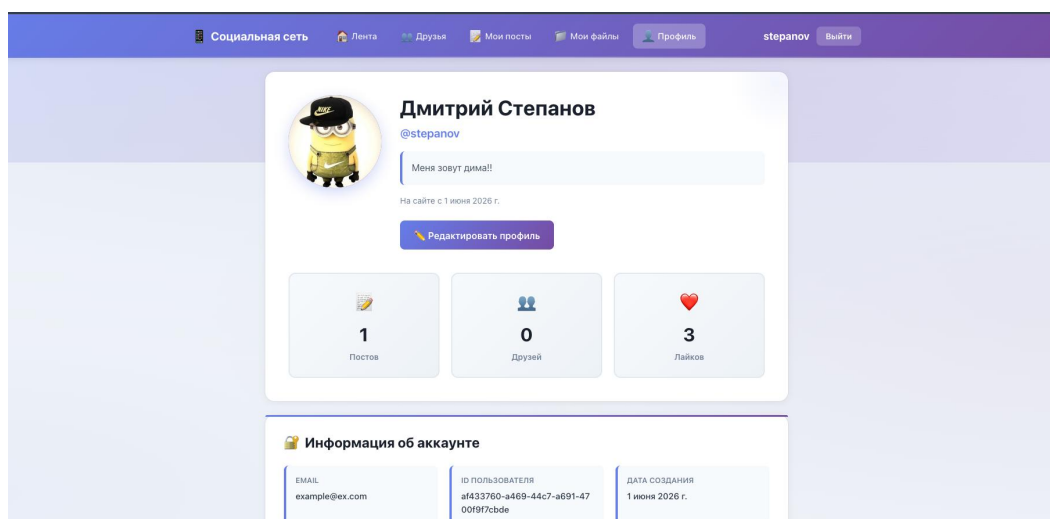


Рисунок Б.5 – Страница профиля

В открывшейся форме укажите ваше имя пользователя, ФИО и загрузите фотографию (аватар). Сохраните изменения.

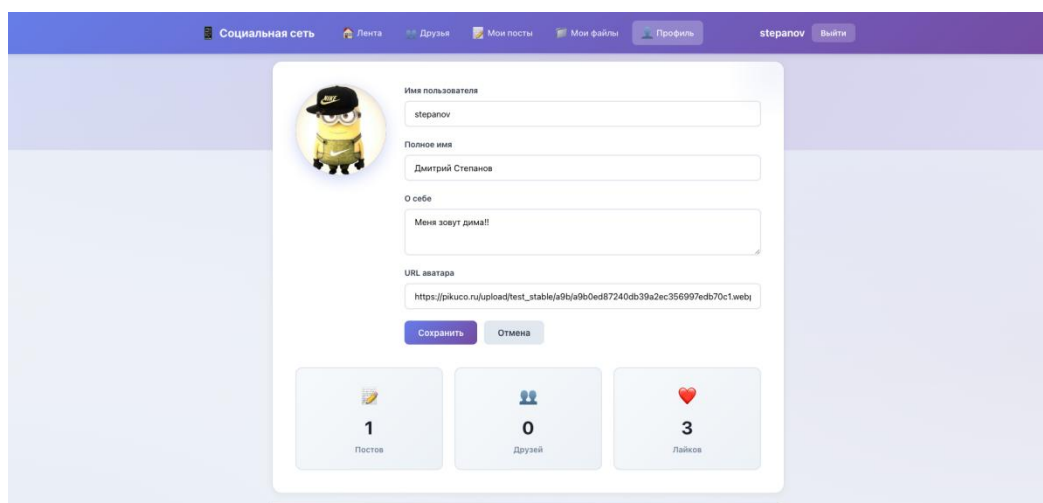
The screenshot shows a web interface for editing a user profile. At the top is a navigation bar with links: 'Социальная сеть', 'Лента', 'Друзья', 'Мои посты', 'Мои файлы', 'Профиль', and a user name 'stepanov' with a 'Выйти' (Logout) button. The main content area is a white form with a purple border. On the left is a circular avatar of a yellow cartoon character wearing a black cap. To the right of the avatar are input fields for 'Имя пользователя' (username) with 'stepanov', 'Полное имя' (full name) with 'Дмитрий Степанов', and 'О себе' (about me) with 'Меня зовут дима!'. Below these is a 'URL аватара' field with a long test URL. At the bottom of the form are 'Сохранить' (Save) and 'Отмена' (Cancel) buttons. Below the form are three summary boxes: '1 Постов' (1 Posts), '0 Друзей' (0 Friends), and '3 Лайков' (3 Likes).

Рисунок Б.6 – Форма редактирования профиля

Б.3.3 Просмотр и создание публикаций (Лента)

Для перехода к новостям выберите раздел «Лента» в навигационном меню.

В верхней части экрана расположено поле «Что у вас нового?». Введите текст публикации и нажмите «Опубликовать».

Ваша запись мгновенно появится в общей ленте новостей.

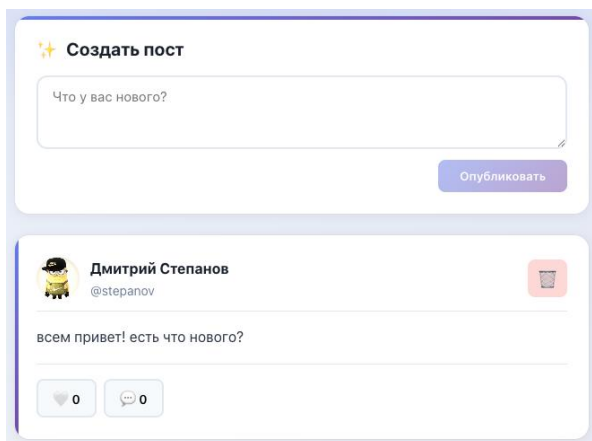
The screenshot shows two parts of a social network interface. The top part is a 'Создать пост' (Create Post) form with a text input field containing 'Что у вас нового?' and an 'Опубликовать' (Publish) button. The bottom part shows a published post by 'Дмитрий Степанов' (@stepanov) with the text 'всем привет! есть что нового?'. The post has 0 likes and 0 comments, indicated by heart and speech bubble icons with the number 0.

Рисунок Б.7 – Опубликованный пост

Б.4 Действия в нештатных ситуациях

Ошибка 401 (Сессия истекла): Ваша сессия безопасности завершилась. Пожалуйста, авторизуйтесь в системе повторно.

Ошибка 500 (Внутренняя ошибка сервера): Сервис временно недоступен в связи с техническими работами. Повторите попытку через несколько минут. При сохранении проблемы обратитесь в службу поддержки по адресу support@student-social.net.